راهنمای جامع پروژهٔ ویسنسباو

WISSENSBAU

از فایل‌های پراکنده تا ویکیِ مرتبط و پاسخ‌های مستند

شرح فارسیِ معماری، داده‌ها، کامپایلر، رابط کاربری، چت، الگوریتم‌های پژوهشی، راه‌اندازی و نگهداری

منبع خام ← پردازش و ترکیب ← ویکی و چت

روش خواندن

چهار صفحهٔ بعد خلاصهٔ پروژه‌اند. از صفحهٔ ۷ به بعد، هر بخش با جزئیات، مثال و مسیر فایل‌های مرتبط توضیح داده می‌شود. فهرست راهنما در صفحهٔ ۶ قرار دارد.

مبنای گزارش: کد و مستندات همین مخزن در تاریخ ۲۴ اکتبر ۲۰۲۶، برابر با ۲ آبان ۱۴۰۵. آخرین commit مشاهده‌شده: c7b5c7c، مورخ ۲۴ سپتامبر ۲۰۲۶.

این متن دربارهٔ یک نمونهٔ پژوهشی غیرتجاری است. نام‌های Labs Aurora و محصولات آن متعلق به داده‌های آزمایشیِ داستانی‌اند. هیچ نتیجه‌ای از مطالعهٔ کاربری ارائه نمی‌شود.

wissensbau.de · Persian project handbook

Sources: AGENTS.md | README.md

خلاصهٔ ۱ | پروژه چه کاری انجام می‌دهد؟

ویسنسباو یک پروژهٔ پژوهشی برای سازمان‌دهی دانش است. ورودی آن می‌تواند یادداشت، متن جلسه، ایمیل، فایل اداری، تصویر، صوت یا دادهٔ واردشده از یک سرویس دیگر باشد. هدف این است که محتوای مرتبطِ چند منبع در صفحه‌های موضوعی جمع شود، صفحه‌ها به هم پیوند بخورند و کاربر بتواند دربارهٔ این مجموعه سؤال بپرسد.

مثلاً توضیح یک حسگر در یادداشت جلسه، ایمیل مهندس و سند مشخصات پخش شده است. سامانه این فایل‌ها را قطعه‌بندی می‌کند، موضوع «حسگر» و موجودیت‌های مرتبط را استخراج می‌کند، یک پیش‌نویس موضوعی می‌نویسد، منابع آن را ثبت می‌کند و صفحه را به موضوع‌هایی مانند باتری و شبکه پیوند می‌دهد. چت سپس بخش‌های مرتبط را پیدا می‌کند و همراه ارجاع پاسخ می‌دهد.

پرسش	پاسخ کوتاه
چه چیزی منبع اصلی است؟	فایل‌های خام در data/raw/؛ ویکی از آن‌ها ساخته می‌شود.
چه چیزی تولید می‌شود؟	Markdown موضوعی، فهرست مطالب، پیوندها و اطلاعات پیگیری پردازش.
هوش مصنوعی کجا نقش دارد؟	استخراج، ترکیب، پیونددهی معنایی و در صورت تنظیم، تولید پاسخ و رتبه‌بندی.
آیا بدون مدل کار می‌کند؟	مرور خروجی موجود و بازیابی واژگانی و پاسخ اقتباسی ممکن است؛ ساخت ویکی به مدل نیاز دارد.
مخاطب فنی پروژه کیست؟	پژوهشگر یا تیم کوچکی که یک مخزن دانش مشترک را مدیریت می‌کند.

> **اصل اصلی**
>
> ویکی یک نمای مشتق‌شده و قابل بازسازی از منابع است. وجود یک جمله در ویکی، به‌تنهایی صحت آن را اثبات نمی‌کند؛ باید منبع و زمینهٔ آن قابل بررسی باشد.

Sources: README.md | compiler/main.py | compiler/rag_engine.py

خلاصهٔ ۲ | تصویر کلی معماری



فرانت‌اند معمولاً روی پورت ۳۰۰۰ و بک‌اند روی ۸۰۰۰ اجرا می‌شود. مرورگر درخواست‌های API را از همان مبدأ فرانت‌اند می‌فرستد؛ فرانت‌اند کوکی نشست را به توکن Bearer برای بک‌اند تبدیل می‌کند. بک‌اند برای کارهای تخصصی، فرایند کوتاه‌عمر Python اجرا می‌کند.

> **تفاوت مهم با متن‌های قدیمی**
>
> در نسخهٔ فعلی، Docusaurus و React برنامهٔ اصلی نیستند. نام wiki-app باقی مانده، اما عمدتاً محل خروجی است. رابط فعلی با Express، EJS، Tailwind و TypeScript ساخته شده است.

Sources: frontend/src/index.ts | frontend/src/config.ts | backend/src/lib/pythonBridge.ts | compiler/models.py

خلاصهٔ ۳ | قابلیت‌ها و میزان اتصال آن‌ها

بخش	آنچه در مخزن وجود دارد	محدودهٔ فعلی
ساخت ویکی	خط لولهٔ پنج‌مرحله‌ای، ساخت افزایشی و کش پاسخ مدل	مدل برای تفسیر و نوشتن لازم است.
چت و جست‌وجو	BM25، embedding، بازرتبه‌بندی، چند معماری RAG	کیفیت هر روش باید جداگانه سنجیده شود.
منابع و اعتماد	جدول ارجاع قطعی و قواعد اعتماد به منبع	امتیاز اعتماد تضمین حقیقت نیست.
موجودیت‌ها	خوشه‌بندی نام‌ها و نمایش موجودیت‌ها	نمای زنده عمدتاً از روش قاعده‌ای استفاده می‌کند.
صف بازبینی	ثبت رأی انسان و بازگرداندن اصلاحات به prompt	نامزدهای صف هنوز از مجموعهٔ پایلوت می‌آیند.
اتصال‌های بیرونی	Gmail، Drive، IMAP، PostgreSQL و SQLite	واردکردن داده با ساخت ویکی دو مرحلهٔ جداست.
عملیات	کاربران، نشست‌ها، تاریخچهٔ اجرا، گزارش و مصرف	مدل دسترسی برای یک تیم مورداعتماد است.
پژوهش	ارزیاب‌ها، مجموعه‌های برچسب‌خورده و پروتکل مطالعه	نتیجهٔ مطالعهٔ انسانی موجود نیست.

دو نوع گراف را باید جدا فهمید: گراف موضوعی از پیوند بین صفحه‌های ویکی ساخته می‌شود؛ گراف ادعاها برای بررسی تأیید، تناقض و جایگزینیِ ادعاهاست. این دو در کد به یک سامانهٔ استنتاج کامل و خودکار تبدیل نشده‌اند.

> **خواندن دقیقِ «پیاده‌سازی شده»**
>
> وجود یک ماژول یا آزمون با مدل مصنوعی، فقط بخشی از آمادگی را نشان می‌دهد. اتصال به رابط، اجرای واقعی با سرویس بیرونی و اثبات کیفیت پژوهشی، هر کدام شواهد جداگانه لازم دارند.

Sources: compiler/rag_architectures.py | compiler/cli.py | documentation/29-active-learning.md | documentation/32-user-study-protocol.md

خلاصهٔ ۴ | کار روزمره و مهم‌ترین محدودیت‌ها

چرخهٔ کار معمول

- کاربر منبع جدید را اضافه یا از یک اتصال وارد می‌کند؛ فایل خام حفظ می‌شود.
- کامپایلر تغییر فایل‌ها را با وضعیت قبلی مقایسه می‌کند و موضوع‌های لازم را بازسازی می‌کند.
- داشبورد نتیجه، مرحلهٔ اجرا و خطاها را نشان می‌دهد؛ ویکی و منابع قابل بررسی می‌شوند.
- چت شواهد مرتبط را بازیابی می‌کند و پاسخ تولیدی یا اقتباسی برمی‌گرداند.
- کاربر پیوند خراب، ادعای مشکوک یا خروجی نامناسب را بررسی و اصلاح ثبت می‌کند.

پنج نکته که باید از ابتدا بدانید

نکته	پیامد عملی
خروجی تولیدشده قابل بازنویسی است	ویرایش مستقیم یک صفحه ممکن است در کامپایل بعدی از بین برود.
تنظیم جدید همیشه فایل را تغییر نمی‌دهد	ساخت افزایشی عمدتاً به تغییر محتوای منبع حساس است؛ برای اعمال سراسری تنظیمات بازسازی لازم می‌شود.
حریم خصوصی یک ویژگی سراسریِ تضمین‌شده نیست	پوشاندن PII در مسیر استخراج متن وصل شده؛ سایر مسیرهای ارسال را جدا باید بررسی کرد.
داده‌های نمونه، واقعی نیستند	عدد باتری یا ویژگی محصول در مثال‌ها نتیجهٔ پژوهش ویسنسباو محسوب نمی‌شود.
سایت عمومی در این بررسی آزموده نشده	گزارش از مخزن محلی تهیه شده؛ وضعیت زندهٔ سرور یا حساب‌ها را تأیید نمی‌کند.

ترتیب مفید برای شناخت فنی: ابتدا پوشه‌ها و معماری، سپس کامپایلر و پیوندها، بعد RAG و اعتماد، و در پایان اجرا، آزمون و محدودیت‌ها. این ترتیب علت وجود هر ماژول را روشن می‌کند.

Sources: compiler/main.py | compiler/synthesizer.py | AGENTS.md

فهرست و راهنمای استفاده از گزارش

بخش	صفحه
خلاصه‌های آغازین	۲ تا ۵
معماری و ورودی‌ها	۷ تا ۱۰
کامپایل، کش و پیوند	۱۱ تا ۱۴
اعتماد و بازبینی	۱۵ تا ۱۸
چت، بازیابی و ذخیره‌سازی	۱۹ تا ۲۲
رابط، بک‌اند و دسترسی	۲۳ تا ۲۷
اتصال‌ها، وب و تنظیمات	۲۸ تا ۳۱
عملیات و اجرا	۳۲ تا ۳۵
آزمون و مطالعهٔ کاربری	۳۶ تا ۳۷
محدودیت و مثال	۳۸ تا ۳۹
عیب‌یابی و نقشهٔ ماژول‌ها	۴۰ تا ۴۱
واژه‌نامه و منابع	۴۲ تا ۴۳

معنای منابع پایین صفحه

مسیرهای کوچک انتهای هر صفحه، فایل‌های همان مخزنند که توضیح آن صفحه بر آن‌ها تکیه دارد. همهٔ مسیرها نسبت به ریشهٔ پروژه خوانده می‌شوند. فهرست منابع منتخب و شیوهٔ بررسی در انتهای گزارش آمده است.

خوانندهٔ غیرفنی می‌تواند صفحات خلاصه، مثال سرتاسری و واژه‌نامه را بخواند. توسعه‌دهنده بهتر است مسیر فایل‌های پایین صفحه را هم دنبال کند. اعداد شمارش مخزن مربوط به زمان بررسی‌اند و با توسعهٔ بعدی تغییر می‌کنند.

۱ | هویت پروژه و مسئلهٔ پژوهشی

نام پروژه Wissensbau است؛ در دستورالعمل مخزن به‌عنوان پژوهشی غیرتجاری با نشانی wissensbau.de معرفی شده است. هستهٔ آن الگوی «ویکی ساخته‌شده از منابع با کمک مدل زبانی» است: انسان مواد خام را نگه می‌دارد و ماشین ساختار و ترکیب دانش را تولید می‌کند. این تقسیم مسئولیت برای ردیابی خطا اهمیت دارد؛ بازنویسی خودکار ورودی می‌تواند شواهدِ علت یک نتیجه را حذف کند.

مسئله فقط پیدا کردن یک واژه نیست. یک موضوع ممکن است در چند قالب و چند زمان مطرح شده باشد، نام یک شخص چند شکل داشته باشد و دو سند دربارهٔ یک ویژگی اختلاف داشته باشند. پروژه می‌کوشد هم دانش را قابل مرور کند، هم پاسخ را به شواهد متصل نگه دارد و هم تعارض‌ها و عدم قطعیت را برای بررسی آشکار سازد.

دامنهٔ نمونه چگونه به پژوهش کمک می‌کند؟

Labs Aurora یک شرکت داستانی در حوزهٔ حسگرهای IoT است. Widget Nova و MeshSync محصولات و فناوری‌های نمونه‌اند. TeaBuddy، Health Nova و Energy GreenGrid دامنه‌های نمونهٔ دیگری هستند. نام‌های مشابه، ایمیل‌های رفت‌وبرگشتی، نسخه‌های قدیمی مشخصات و اختلاف دربارهٔ باتری، عمداً فرصت آزمودن استخراج، تشخیص هویت و بررسی تعارض را فراهم می‌کنند.

در نتیجه، نباید این مخزن را شرح یک شرکت واقعی یا مجموعهٔ نتایج معتبر دربارهٔ سخت‌افزار تلقی کرد. ارزش داده‌های نمونه در قابل‌کنترل بودن سناریوهاست. تعمیم رفتار الگوریتم به یادداشت‌ها یا ایمیل‌های واقعی، به داده و ارزیابی مستقل نیاز دارد.

> **پرسش‌های باز**
>
> آیا صفحه‌های ترکیبی نسبت به جست‌وجوی مستقیم کمک می‌کنند؟ آیا ارجاع‌ها برای کاربر قابل بررسی‌اند؟ آیا مدل تناقض را حفظ می‌کند یا یکی از طرف‌ها را حذف می‌کند؟ کدام مرحله بیشترین هزینه یا خطا را ایجاد می‌کند؟ این‌ها موضوع ارزیابی‌اند، نه نتیجهٔ ازپیش‌تعیین‌شده.

Sources: AGENTS.md | documentation/18-sample-domain.md | documentation/32-user-study-protocol.md

۲ | نقشهٔ پوشه‌ها و مالکیت داده

مسیر	نقش	نکتهٔ نگهداری
data/raw/	اصل فایل‌های ورودی	متعلق به انسان؛ در تهیهٔ این گزارش تغییر داده نشده است.
compiler/	پردازش Python و الگوریتم‌ها	ورودی اصلی main.py و پل فراخوانی cli.py است.
compiler/temp_output/	پیش‌نویس موضوع‌ها و index.json	مرحلهٔ میانی و قابل بازسازی.
wiki-app/docs/	صفحه‌های نهایی Markdown	خروجی تولیدشده و ورودی اصلی چت ویکی.
wiki-app/static/media/	نسخهٔ قابل ارائهٔ رسانه و پیوست	با هش محتوا از تکرار جلوگیری می‌شود.
backend/	API و سرویس‌های فایل و حساب	Node.js و TypeScript.
frontend/	صفحه‌ها و کد مرورگر	Express، EJS، Tailwind و esbuild.
documentation/	راهنماهای موضوعی	۴۱ فایل شماره‌دار؛ برخی توضیح‌ها تاریخی‌اند.
deploy/	پیکربندی و اسکریپت استقرار	در کنار فایل‌های Docker Compose.

پوشهٔ data فقط ورودی نیست. وضعیت پردازش، تاریخچهٔ چت، کاربران و نشست‌ها، تنظیمات، مصرف و اجرای خط لوله نیز در آن قرار می‌گیرند. بنابراین نسخهٔ پشتیبانِ تنها صفحه‌های ویکی، برای بازیابی کامل برنامه کافی نیست. از طرف دیگر، پوشهٔ temp_output نباید با خروجی نهایی اشتباه گرفته شود.

در نسخهٔ بررسی‌شده ۲۲۷ فایل Markdown در ریشهٔ خروجی، ۵۸ ماژول Python در ریشهٔ compiler، ۷۳ فایل آزمون Python و ۲۱ فایل آزمون بک‌اند وجود داشت. ورودی محلی شامل ۸۳ فایل بود: ۲۴ Markdown، ۴۳ متن و ۱۶ ایمیل. این اعداد موجودی فایل‌اند، نه شاخص کیفیت یا نتیجهٔ آزمون.

Sources: compiler/models.py | backend/src/paths.ts | documentation/10-data-layout-and-state.md

۳ | معماری اجرایی و مسیر یک درخواست

سامانه از دو سرویس دائمی Node و چند فرایند کوتاه‌عمر Python تشکیل می‌شود. فرانت‌اند HTML را با قالب‌های EJS تولید می‌کند. بک‌اند قواعد API، دسترسی به فایل و حساب‌ها را مدیریت می‌کند. Python برای کارهایی باقی مانده که از قبل منطق تخصصی دارند: کامپایل، بازیابی و تولید پاسخ، تجزیهٔ ایمیل، موجودیت‌ها، بازبینی و اتصال‌ها.

```
Browser
  -> frontend :3000
    -> /api proxy + session cookie -> Bearer
      -> backend :8000
        -> python3 cli.py <command>
        -> python3 -u main.py [options]
          -> data/ + wiki-app/docs/
```

برای نمونه، صفحهٔ چت درخواست را به API فرانت‌اند می‌فرستد. proxy، توکن کوکی HttpOnly را به هدر Authorization تبدیل می‌کند. بک‌اند احراز هویت را بررسی می‌کند و cli.py را با ورودی JSON اجرا می‌کند. خروجی معمول یک شیء JSON است؛ در چت جریانی، رویدادها خط‌به‌خط تولید و به مرورگر منتقل می‌شوند.

چرا ارتباط با Python یک سرویس جدا نیست؟

پل subprocess نیاز به نگهداری یک API دیگر را کم می‌کند و منطق موجود Python را حفظ می‌کند. هزینهٔ آن، راه‌اندازی مکرر مفسر و لزوم مدیریت stdout، stderr، خطا و همزمانی است. Semaphore تعداد فراخوانی‌های کوتاه Python را محدود می‌کند؛ ساخت ویکی جداگانه سری‌سازی می‌شود و در پل فعلی حداکثر یک ساخت دیگر در صف می‌ماند.

> **دو آدرس متفاوت**
>
> BACKEND_API_URL برای تماس داخل سرور است و در Docker می‌تواند نام backend باشد. PUBLIC_API_URL در نسخهٔ دارای ورود، به‌طور پیش‌فرض خالی است تا مرورگر از proxy همان مبدأ استفاده کند. راهنمای قدیمی تماس مستقیم مرورگر با پورت ۸۰۰۰، مرجع وضعیت فعلی نیست.

Sources: frontend/src/index.ts | frontend/src/config.ts | backend/src/lib/pythonBridge.ts | compiler/cli.py

۴ | ورودی‌ها، رسانه و تبدیل به متن

همهٔ قالب‌ها در نهایت به RawChunk تبدیل می‌شوند. هر قطعه، متن، مسیر منبع، شمارهٔ قطعه و نوع منبع دارد. این یکسان‌سازی اجازه می‌دهد مراحل بعدی برای ایمیل و PDF و یادداشت، مسیر مشترکی داشته باشند. بااین‌حال کیفیت متنِ حاصل برای همهٔ قالب‌ها یکسان نیست.

قالب	رفتار واقعی	حد یا پیش‌شرط
MD / TXT	خواندن متن و شکستن در مرز پاراگراف	اندازهٔ هدف قطعه حدود ۲۰۰۰ نویسه است.
EML	تجزیهٔ هدر، بدنه و پیوست‌ها	text/plain ترجیح دارد؛ HTML به متن تبدیل می‌شود.
تصویر	توصیف تصویر با مدل و لینک خود فایل	به مدل دارای توانایی تصویر نیاز دارد.
صوت	رونویسی و لینک شنیدن یا دریافت	در خطا ممکن است فقط فراداده باقی بماند.
PDF	استخراج لایهٔ متن با pypdf	OCR عمومی برای PDF اسکن‌شده پیاده نشده است.
PPTX / XLSX / DOCX	پاراگراف و جدول / سلول / متن و یادداشت اسلاید	XLSX به ۱۰۰ ردیف در هر شیت محدود می‌شود.
XML / JSON / CSV و مشابه	استخراج یا نمایش متنی ساختار	این کار تحلیل معنایی کامل داده نیست.
ZIP	فهرست نام و اندازهٔ اعضا	محتوا به‌طور بازگشتی استخراج نمی‌شود.
ویدئو و قالب‌های opaque	اطلاعات فایل و لینک دریافت	وجود فایل به معنی فهم محتوای آن نیست.

رسانه و پیوست در static/media کپی می‌شود و هش محتوا جلوی نسخه‌های تکراری را می‌گیرد. برخی parserها در خرابی فایل یا نبود کتابخانه، به پیوست بدون استخراج محتوا تنزل می‌کنند. بنابراین «پذیرفته شدن قالب» را باید از «متن باکیفیت برای دانش‌سازی» جدا دانست.

Sources: compiler/media_ingest.py | compiler/email_ingest.py | compiler/all_source_extensions.json | documentation/19-multimedia-email-and-trust.md

۵ | مرحله‌های اول و دوم: خواندن و استخراج

مرحلهٔ اول: کشف و قطعه‌بندی

کشف فایل زیر data/raw به‌صورت بازگشتی انجام می‌شود و پسوندهای شناخته‌شده را انتخاب می‌کند. متن در مرز پاراگراف‌ها به قطعه‌های کوچک‌تر تقسیم می‌شود تا ورودی مدل قابل مدیریت باشد. فایل غیرمتنی ابتدا از handler ویژهٔ خود می‌گذرد. شناسهٔ منبع و شمارهٔ قطعه حفظ می‌شود تا بعداً معلوم باشد هر موضوع از کجا آمده است.

مرحلهٔ دوم: ساخت اطلاعات ساخت‌یافته

مدل برای هر قطعه فهرست موضوع‌ها، موجودیت‌ها و مفهوم‌ها را به شکل JSON برمی‌گرداند. موضوع مسیر ساخت صفحه را تعیین می‌کند؛ موجودیت معمولاً شخص، شرکت یا محصول است؛ مفهوم می‌تواند اصطلاحی مانند مصرف توان یا شبکهٔ مش باشد. یک قطعه ممکن است به چند موضوع مربوط باشد و در ترکیب چند صفحه استفاده شود.

```
{
  "topics": ["MeshSync", "Battery"],
  "entities": [{"name": "Mira Chen", "description": "..."}],
  "concepts": [{"name": "Power Management", "description": "..."}]
}
```

نتیجه همراه MD5 فایل، متن قطعه و زمان پردازش در state.json ذخیره می‌شود. این فایل حافظهٔ عملیاتی کامپایلر است؛ صرفاً گزارشی برای نمایش نیست. متن ذخیره‌شده محلی می‌تواند اصل متن بدون پوشاندن اطلاعات حساس باشد، حتی وقتی متن prompt استخراج پالایش شده است.

> **چرا خطای استخراج مهم است؟**
>
> اگر دو نام یکسان اشتباه جدا شوند یا یک موضوع اصلاً استخراج نشود، مراحل ترکیب و پیوند نیز اثر آن را می‌گیرند. منبع ممکن است موجود باشد، اما به صفحهٔ مناسب نرسد. آزمون قالب JSON به‌تنهایی کیفیت این تصمیم معنایی را اثبات نمی‌کند.

Sources: compiler/main.py | compiler/synthesizer.py | compiler/text_chunking.py

۶ | مرحلهٔ سوم: ترکیب صفحهٔ موضوعی

پس از استخراج، قطعه‌ها بر اساس نام موضوع گروه‌بندی می‌شوند. یک صفحه می‌تواند از چند فایل و چند نوع منبع تغذیه شود. این تفاوت اصلی با خلاصه‌کردن تک‌فایل است: خروجی قرار است دانشِ یک موضوع را جمع کند، نه ترتیب پوشه‌های ورودی را تکرار کند. قطعهٔ بدون موضوع می‌تواند در گروه Notes General قرار بگیرد.

برای هر موضوعی که باید بازسازی شود، prompt شامل نام موضوع، شناسهٔ پیشنهادی، برچسب‌ها، زمان و بلوک‌های منبع است. مدل Markdown می‌نویسد و نتیجه به نامی مانند meshsync.md در temp_output ذخیره می‌شود. slugify نام موضوع را برای استفاده در نام فایل و شناسه نرمال می‌کند؛ خروجی فعلی عمدتاً تخت و موضوع‌محور است.

منابع چگونه به متن وصل می‌شوند؟

مدل مأمور نوشتن فهرست منابع نهایی نیست. پس از پاسخ، بخش منابع احتمالیِ نوشته‌شده توسط مدل حذف و جدول Trust & References با کد ساخته می‌شود. در این جدول، هر منبع یک بار ثبت می‌شود. این روش از اختراع نام فایل در فهرست ارجاع جلوگیری می‌کند، اما همچنان نمی‌گوید هر جمله دقیقاً از کدام قطعه پشتیبانی می‌گیرد.

اصلاحات و منتقد

در کد فعلی، extra_system_context می‌تواند اصلاحات انسان را به prompt ترکیب و منتقد اضافه کند. منتقد اختیاری، پیش‌نویس را با همان منابع مقایسه می‌کند. بنابراین این مرحله هم محل اصلی یکپارچه‌سازی است و هم نقطه‌ای که باید ساختن جزئیاتِ بی‌پشتوانه را سنجید.

> **ویرایش خروجی**
>
> تغییر دستی فایل نهایی، ورودی پایدار ساخت بعدی محسوب نمی‌شود. اگر موضوع دوباره ساخته یا پیونددهی شود، تغییر ممکن است بازنویسی شود. وجود ویرایشگر در رابط این رفتار چرخهٔ تولید را عوض نمی‌کند.

Sources: compiler/synthesizer.py | compiler/trust.py | compiler/active_learning.py | frontend/src/routes/wiki.ts

۷ | ساخت افزایشی، وضعیت و کش مدل

کامپایلر هر بار همهٔ کار را از ابتدا انجام نمی‌دهد. MD5 محتوای فایل با مقدار ذخیره‌شده مقایسه می‌شود و فایل‌ها به جدید، تغییرکرده، حذف‌شده و بدون تغییر تقسیم می‌شوند. این MD5 ابزار تشخیص تغییر است، نه سازوکار امنیتی. در ساخت معمول، استخراج فایل‌های بدون تغییر قابل استفادهٔ مجدد است.

لایهٔ صرفه‌جویی	چه چیزی را مقایسه می‌کند؟	اثر
وضعیت فایل	هش فایل و رکورد قبلی	از استخراج تکراری جلوگیری می‌کند.
موضوع‌های نیازمند بازسازی	موضوع‌های وابسته به منابع تغییرکرده	دامنهٔ ترکیب را محدود می‌کند.
بازپیونددهی	پیش‌نویس و تغییر نمایه و قواعد پیوند	فقط خروجی‌های مؤثر را دوباره می‌نویسد.
کش SQLite مدل	کلید دقیق درخواست	پاسخ قبلی درخواست همسان را برمی‌گرداند.

کلید متنی کش در کد بررسی‌شده از prompt system، prompt، نام مدل و temperature ساخته می‌شود. تصویر و صوت کلیدهای محتوایی ویژه دارند. تغییر کوچک prompt ممکن است کش را بی‌اثر کند؛ مثلاً زمان جدید در prompt ترکیب می‌تواند درخواست متفاوتی بسازد. بنابراین حتی force-- هم لزوماً همه‌جا پاسخ کش‌شده نمی‌گیرد و نباید آن را اجرای بدون هزینه فرض کرد.

گزینهٔ force-- فایل‌ها و موضوع‌ها را بدون اتکا به تشخیص عدم تغییر بازپردازش می‌کند. تغییر تنظیمات مدل، منتقد یا اصلاحات انسانی لزوماً هش فایل خام را تغییر نمی‌دهد؛ برای اثرگذاری بر همهٔ خروجی‌های موجود، باید بازسازی مناسب برنامه‌ریزی شود. حذف یک منبع نیز باید از نظر صفحه‌ها و پیوندهای باقی‌مانده بررسی شود.

> **دو چیز متفاوت**
>
> state.json حافظهٔ استخراج و وابستگی داده است. llm-cache.sqlite حافظهٔ پاسخ درخواست‌های مدل است. حذف یکی، معادل حذف دیگری نیست و بر سرعت، هزینه و بازتولیدپذیری اثر متفاوت می‌گذارد.

Sources: compiler/synthesizer.py | compiler/main.py | compiler/llm_client.py | documentation/10-data-layout-and-state.md

۸ | نمایه، پیونددهی و فهرست مطالب

مرحلهٔ چهارم نگاشت عنوان موضوع به نام فایل را در index.json می‌سازد یا به‌روزرسانی می‌کند. این نمایه مرجع مرحلهٔ پیونددهی است. بدون آن، مدل نمی‌تواند با اطمینان بداند یک اشاره باید به کدام فایل وصل شود. تغییر عنوان یا حذف موضوع نیز باید در نمایه منعکس شود.

پیونددهی دو لایه دارد

ابتدا mechanical_linker اشاره‌های دقیق و بدون ابهام به عنوان‌ها را پیوند می‌کند. این گذر از کد، عنوان‌ها و پیوندهای موجود محافظت می‌کند و از تطبیق‌های ناخواسته می‌کاهد. نام‌های جایگزین حاصل از تشخیص موجودیت نیز می‌توانند به نمایهٔ این گذر افزوده شوند. سپس گذر مدل برای اشاره‌های غیرمستقیم یا بازنویسی‌شده اجرا می‌شود؛ وجود بخش مکانیکی، کامپایلر را به یک مسیر بدون مدل تبدیل نکرده است.

قواعد دستی در link_overrides.json می‌توانند رابطه‌ای را الزامی یا مسدود کنند. این قواعد پس از تصمیم‌های خودکار اعمال می‌شوند و برای اصلاح پایدارتر ساختار رابطه مفیدند. شکل wikilink مانند [[topic|Label]] نیز در مسیر خروجی به پیوند Markdown تبدیل می‌شود.

خروجی نهایی و MOC

صادرکننده matter front را کامل و متن را برای سازگاری با قراردادهای قدیمی MDX پاک‌سازی می‌کند. یادداشت AUTO-GENERATED مشخص می‌کند فایل تولیدشده است. سپس moc_generator فهرست سلسله‌مراتبی index.md را می‌سازد. دسته‌بندی از پوشه‌ها و برچسب‌های پرکاربرد مجموعه استفاده می‌کند؛ دسته‌های آن قرار نیست فقط برای Aurora تنظیم شده باشند.

> **معنای یک پال**
>
> پیوند «باتری ← محصول» معمولاً فقط رابطهٔ ناوبری یا اشارهٔ متنی است. آن را نباید خودبه‌خود به معنای رابطهٔ علّی، تأیید ادعا یا هم‌هویتی دو موجودیت تفسیر کرد.

Sources: compiler/linker.py | compiler/mechanical_linker.py | compiler/link_overrides.py | compiler/moc_generator.py

۹ | منابع، اعتماد و قابلیت ردیابی

trust.py اعتماد اولیه به منبع را به‌صورت قطعی تعیین می‌کند. ابتدا قواعد glob روی مسیر نسبی بررسی می‌شوند و اولین قاعدهٔ منطبق اولویت دارد. سپس مقدار پیش‌فرض نوع منبع و در آخر پیش‌فرض داخلی استفاده می‌شود. سطح‌ها unverified، low، medium، high و verified هستند؛ این نام‌ها تصمیم سیاستی سامانه‌اند، نه گواهی مستقل واقعیت.

نوع منبع	پیش‌فرض داخلی	تفسیر
متن، ایمیل و فایل	medium	متن قابل خواندن است؛ درستی آن هنوز باید سنجیده شود.
تصویر و صوت	low	توصیف یا رونویسی می‌تواند خطای مدل داشته باشد.
نتیجهٔ وب	low	شاهد بیرونی هنوز بررسی مستقل نشده است.
دادهٔ داستانیِ مشخص‌شده در قواعد	unverified	قاعدهٔ مسیر می‌تواند پیش‌فرض نوع را جایگزین کند.

Resources وارونِ جدول ارجاع را می‌سازد: به جای اینکه فقط بپرسیم «این صفحه از چه استفاده کرده؟»، می‌پرسیم «این منبع در کدام صفحه‌ها استفاده شده؟». منابع با مسیر یکسان ادغام می‌شوند و صفحه‌های ارجاع‌دهنده کنار آن قرار می‌گیرند. این برای ارزیابی اثر اصلاح یا حذف یک منبع مفید است.

سه پرسش مستقل

- قابلیت ردیابی: آیا می‌دانیم متن از کدام فایل‌ها آمده است؟
- پشتیبانی: آیا منابع واقعاً جملهٔ خروجی را تأیید می‌کنند؟
- صحت: آیا خودِ آن منابع ادعای درستی دارند؟

جدول قطعی عمدتاً پرسش اول را پاسخ می‌دهد. منتقد برای پرسش دوم ساخته شده و انتشار اعتماد بخشی از پرسش سوم را با ساختار ادعا بررسی می‌کند. یکی جای دیگری را نمی‌گیرد و ارجاع زیاد هم لزوماً چند شاهد مستقل نیست.

Sources: compiler/trust.py | compiler/resources_engine.py | backend/src/lib/resourcesEngine.ts

۱۰ | انتشار اعتماد روی گراف ادعاها

اعتماد ثابت به فایل نمی‌تواند تفاوت دو ادعا در همان فایل را بیان کند. مثلاً یک صورت‌جلسه ممکن است نوع باتری را درست و فاصلهٔ نمونه‌برداری را غلط نوشته باشد. trust_propagation برای هر ادعا یک امتیاز جدا می‌سازد و از رابطهٔ تأیید، تناقض و جایگزینی استفاده می‌کند.

ابتدا هر ادعا prior خود را از قواعد اعتماد منبع می‌گیرد. در هر دور، امتیاز همسایه‌های تأییدکننده حمایت را زیاد و امتیاز طرف‌های متناقض یا جایگزین‌کننده آن را کم می‌کند. تناقض متقارن است؛ جایگزینی جهت دارد و به ادعای قدیمی آسیب می‌زند. یک ضریب کاهش جدا نیز برای ادعای جایگزین‌شده اعمال می‌شود.

```
support = corroboration - contradiction - supersession
score = clamp01((alpha * prior
      + (1 - alpha) * sigmoid(support)) * decay)
```

به‌روزرسانی‌ها از امتیاز دور قبل استفاده می‌کنند. مقدار پیش‌فرض prior_weight برابر ۰٫۲ است؛ در اسناد، انتخاب آن بر پایهٔ پایلوت توضیح داده شده است. امتیاز حاصل احتمال کالیبره‌شدهٔ حقیقت نیست. الگوریتم gold_label را نباید بخواند؛ آن برچسب فقط برای ارزیابی خروجی است.

داده و محدودهٔ واقعی

مجموعهٔ پایلوت فعلی ۶ گروه، ۲۹ ادعا و ۲۹ رابطه دارد. برچسب‌ها شامل درست، نادرست، جایگزین‌شده، وابسته به شرایط و محل اختلاف‌اند. مستندات آزمایش‌های آفلاین و حذف مؤلفه‌ها را شرح می‌دهند؛ این‌ها مطالعهٔ انسانی نیستند و در تهیهٔ این PDF دوباره اجرا نشده‌اند. شمار کوچک، دامنهٔ داستانی و نبود رابطهٔ بین گروه‌ها، تعمیم را محدود می‌کند.

> **جایگاه در محصول**
>
> جدول اعتماد زندهٔ منابع همچنان از قواعد ثابت می‌آید. ماژول انتشار و GraphStore مسیر فراخوانی دارند، اما این به معنای استخراج خودکار گراف ادعا برای همهٔ منابع در کامپایل عادی نیست.

Sources: compiler/trust_propagation.py | compiler/trust_eval_dataset.py | data/trust_eval_dataset.json | documentation/23-trust-propagation-evaluation.md

۱۱ | موجودیت‌ها و زمان: دو نوع رفع ابهام

تشخیص هویت یا Resolution Entity

Mira، Chen Mira و شکل ایمیلی نام ممکن است یک شخص باشند. entity_resolution ابتدا تطبیق دقیق، زیرمجموعهٔ واژه‌های نام و الگوی نام در ایمیل را بررسی می‌کند. حالت‌های مبهم می‌توانند به embedding یا داوری مدل برسند. خوشه‌بندی از union-find استفاده می‌کند و نام کامل خوانا را برای نمایش ترجیح می‌دهد.

اشتراک یک نام کوچک کافی نیست: Kim Alex و Rivera Alex نباید فقط به دلیل Alex بودن ادغام شوند. بااین‌حال یک نام بسیار کوتاه مثل Alex در دادهٔ واقعی همچنان می‌تواند مبهم باشد. نتیجهٔ خوب روی پایلوت کوچک، حل همهٔ هم‌نامی‌ها و لقب‌ها را ثابت نمی‌کند. entity_graph، اشاره‌های state.json را می‌خواند و نمای فعلیِ موجودیت‌ها را عمدتاً با لایهٔ قاعده‌ای و بدون هزینهٔ مدل می‌سازد.

مدل زمانی یا Modeling Temporal

temporal_model می‌پرسد «در یک تاریخ، کدام ادعا معتبر جاری شمرده می‌شد؟». بازه از valid_from شروع می‌شود و در تاریخ ادعای جایگزین‌کننده پایان می‌گیرد. current_claims ادعاهای جایگزین‌نشده را برمی‌گرداند و as_of یک تاریخ مشخص می‌گیرد. تناقض صرف، بدون رابطهٔ supersedes، به معنی پایان اعتبار زمانی نیست.

مفهوم	معنا
زمان اعتبار	دوره‌ای که مقدار ادعا برای آن جاری فرض می‌شود.
زمان ثبت	زمانی که سامانه از ادعا باخبر یا آن را ثبت کرده است.
محدودیت فعلی	recorded_at وجود دارد، ولی در دادهٔ نمونه جدایی واقعی دو زمان نشان داده نشده است.

این دو ماژول خطاهای متفاوتی را هدف می‌گیرند: تشخیص هویت از مخلوط‌شدن اشخاص و محصولات می‌کاهد؛ زمان از مخلوط‌شدن نسخهٔ قدیمی و جدید. endpoint زمانی وجود دارد، اما دادهٔ آن هنوز از ساختار پایلوت می‌آید و یک موتور زمانیِ کامل برای همهٔ چت‌ها نیست.

Sources: compiler/entity_resolution.py | compiler/entity_graph.py | compiler/temporal_model.py | compiler/cli.py

۱۲ | منتقد، بازبینی انسان و حلقهٔ اصلاح

منتقد خروجی ترکیب

extraction_critic پیش‌نویس را همراه منابع به یک گذر دیگر مدل می‌دهد تا جمله‌های بی‌پشتوانه را مشخص کند. کد فقط جمله‌ای را حذف می‌کند که بتواند عیناً در پیش‌نویس پیدا کند؛ اگر منتقد جمله را بازنویسی کرده باشد، حذف مبهم انجام نمی‌شود. پاسخ خراب با parse_error ثبت می‌شود و سازوکار منتقد به‌تنهایی نباید ساخت را متوقف کند.

با critic-samples-- چند داوری و رأی اکثریت ممکن است. با critic-regenerate--، اگر حذف‌ها بیش از آستانهٔ پیش‌فرض ۲۰ درصد طول متن باشد، یک بار بازتولید با بازخورد انجام می‌شود. این حلقه محدود است. تکرار یک داور خطای تصادفی را کم می‌کند، اما سوگیری مشترک مدل را تضمین‌شده اصلاح نمی‌کند.

صف بازبینی انسان

active_learning ادعاهای کم‌اعتماد یا تناقض‌های حل‌نشده را انتخاب می‌کند. آستانهٔ پیش‌فرض کم‌اعتمادی ۰٫۳۵ و فاصلهٔ تناقض حل‌نشده ۰٫۱۵ است. کاربر در /review-queue می‌تواند درست، نادرست، جایگزین‌شده یا وابسته به شرایط بودن را ثبت کند و یادداشت بگذارد. اصلاحات بر اساس شناسهٔ ادعا ذخیره و جایگزین می‌شوند.

پرچم use-corrections-- اصلاحات ذخیره‌شده را به مثال‌های prompt تبدیل می‌کند. کد فعلی این زمینه را به استخراج، ترکیب و منتقد می‌رساند. این روش، راهنمایی مدل در درخواست‌های بعدی است؛ آموزش وزن‌های مدل یا تغییر مستقیم منابع خام نیست. برای اثر روی فایل‌های بدون تغییر، رفتار ساخت افزایشی باید در نظر گرفته شود.

> **محدودیت تعیین‌کننده**
>
> صف فعلی از trust_eval_dataset.json نامزد می‌سازد. هنوز نمی‌توان نتیجه گرفت همهٔ ادعاهای هر فایل واردشده به‌طور خودکار بررسی و وارد صف می‌شوند. اتصال صف به گراف ادعاهای استخراج‌شده از مجموعهٔ واقعی، کار باقی‌مانده است.

Sources: compiler/extraction_critic.py | compiler/active_learning.py | compiler/synthesizer.py | frontend/src/client/review-queue.ts

۱۳ | چت RAG از سؤال تا پاسخ

RAG یعنی پیش از نوشتن پاسخ، شواهد مرتبط از مجموعه بازیابی شوند. در مسیر اصلی، corpus از صفحه‌های تولیدشدهٔ ویکی ساخته می‌شود. هر صفحه بر اساس عنوان بخش‌ها به passageهای کوچک‌تر تقسیم می‌شود؛ بنابراین واحد بازیابی الزاماً کل فایل نیست. عنوان صفحه و بخش، مسیر و متن همراه passage می‌مانند.

سؤال	بازیابی	گزینش شواهد	پاسخ و ارجاع

answer_question ابتدا تنظیمات و دامنهٔ انتخاب‌شده را می‌خواند، passageها را رتبه‌بندی می‌کند و top_k مورد را برای پاسخ آماده می‌سازد. مدل در حالت تولیدی باید فقط از این متن‌ها استفاده و با شماره‌هایی مانند [1] ارجاع دهد. ارجاع‌ها همراه نتیجه به رابط می‌رسند تا قابل بررسی باشند.

حالت پاسخ	معنا و محدودیت
generated	متن تازه با مدل نوشته شده؛ نیازمند بررسی پشتیبانی ادعاهاست.
extractive	عنوان و قطعه‌هایی از متن بازیابی‌شده ارائه می‌شود؛ تولید آزاد ندارد.
empty	مجموعهٔ مناسب برای جست‌وجو موجود نیست.
no_match	مدرک مرتبط کافی برای سؤال پیدا نشده است.

در کد فعلی، علاوه بر ویکی، corpus_source می‌تواند raw باشد و build_raw_corpus مسیر سؤال از منابع خام را فراهم می‌کند. دامنهٔ doc_scope برای انتخاب صفحه‌های ویکی معنا دارد و نباید آن را کنترل دسترسی سطح سند فرض کرد. تاریخچه و نشست‌های چت نیز در بک‌اند ذخیره و پاسخ جریانی پشتیبانی می‌شوند.

> **برداشت صحیح از پاسخ اقتباسی**
>
> اقتباس لفظ‌به‌لفظ، ساختن جملهٔ تازه را محدود می‌کند؛ اما اگر متن منبع غلط یا بازیابی نامرتبط باشد، پاسخ همچنان ممکن است برای سؤال درست نباشد. وفاداری به متن و صحت پاسخ دو معیار جدا هستند.

Sources: compiler/rag_engine.py | compiler/rag_types.py | compiler/cli.py | backend/src/lib/chatSessions.ts

۱۴ | بازیابی ترکیبی: واژه، معنا و بازرتبه‌بندی

۱. جست‌وجوی واژگانی BM25

BM25 هم‌پوشانی واژه‌های سؤال و passage را می‌سنجد. واژهٔ کم‌تکرار اطلاعات بیشتری دارد؛ تکرار یک واژه سود نامحدود نمی‌دهد و طول متن نیز در امتیاز دخالت می‌کند. این لایه بدون API و embedding قابل اجراست. پارامترهای k1 و b در تنظیمات قابل تغییرند؛ پیش‌فرض کد به‌ترتیب ۱٫۵ و ۰٫۷۵ است.

۲. بردار معنایی و ادغام رتبه‌ها

embedding نمایش عددی متن است. سؤال و passage به بردار تبدیل می‌شوند و شباهت کسینوسی آن‌ها سنجیده می‌شود. این لایه ممکن است رابطهٔ معنایی واژه‌های متفاوت را پیدا کند. چون عدد BM25 و کسینوس مقیاس یکسانی ندارند، RRF رتبه‌ها را ادغام می‌کند، نه اینکه خامِ دو امتیاز را جمع کند.

```
RRF(document) = sum(1 / (constant + rank_in_list))
```

۳. بازرتبه‌بندی با مدل

مدل shortlist را به ترتیب ارتباط با سؤال مرتب می‌کند. پاسخ نامعتبر یا شکست فراخوانی باید به ترتیب قبلی برگردد. اگر embedding شکست بخورد، BM25 می‌تواند ادامه دهد؛ اگر بازرتبه‌بندی شکست بخورد، رتبهٔ ادغام‌شده حفظ می‌شود. چنین تنزلی، دسترس‌پذیری را بیشتر می‌کند ولی ممکن است کیفیت را تغییر دهد.

top_k تعداد متن‌های نهایی را تعیین می‌کند. مقدار بیشتر ممکن است پوشش را زیاد کند، اما شواهد نامربوط، تأخیر و هزینهٔ prompt را هم افزایش می‌دهد. انتخاب آن باید با سؤال‌ها و اسناد همان مجموعه سنجیده شود. گزارش‌های پایلوت در مخزن، برتری عمومیِ هر لایه را اثبات نمی‌کنند و مقایسهٔ مدل واقعی در این گزارش اجرا نشده است.

سه خطای متفاوت

بازیابی می‌تواند سند لازم را پیدا نکند؛ بازرتبه‌بندی می‌تواند سند خوب را پایین ببرد؛ مدل پاسخ می‌تواند حتی از شواهد درست نتیجهٔ نادرست بسازد. ارزیابی باید این مرحله‌ها را جدا بررسی کند.

Sources: compiler/hybrid_retrieval.py | compiler/rag_engine.py | compiler/rag_settings.py | documentation/25-hybrid-retrieval.md

۱۵ | معماری‌های قابل انتخاب RAG

معماری	مداخله در فرایند	بهای احتمالی و حد
Naive	یک بازیابی و سپس پاسخ	مبنای ساده؛ اصلاح سؤال یا بازیابی ندارد.
Hybrid	ادغام واژه و معنا و بازرتبه‌بندی	فراخوانی و هزینهٔ بیشتر در لایه‌های اختیاری.
HyDE	ساخت پاسخ فرضی و embedding آن	پاسخ فرضی شاهد نیست؛ برای هدایت بازیابی است.
RAG-Fusion	چند بازنویسی سؤال و ادغام نتیجه‌ها	پوشش واژگان بیشتر؛ امکان تغییر منظور سؤال.
GraphRAG-lite	گسترش از صفحه‌های اولیه به همسایه‌های پیوندی	از لینک Markdown استفاده می‌کند، نه موتور GraphRAG کامل.
Corrective	ارزیابی ارتباط و تلاش دوباره با سؤال بازنویسی‌شده	نسخهٔ حاضر از بازنویسی و بازیابی مجدد استفاده می‌کند.

HyDE برای فاصلهٔ زبانی بین سؤال کوتاه و پاسخ بلند طراحی شده است. اگر سؤال «چرا باتری خالی می‌شود؟» با واژه‌های سند فرق دارد، متن فرضی ممکن است مسیر معنایی بهتری بسازد. این متن نباید به‌عنوان حقیقت به کاربر ارائه شود؛ شواهد نهایی همچنان باید از corpus بیایند.

در GraphRAG-lite، صفحهٔ ظاهراً کم‌شباهت اما مرتبط با صفحهٔ اصلی از طریق یال‌های ویکی وارد نامزدها می‌شود. تعداد گام‌ها قابل تنظیم و وزن با فاصله کم می‌شود. این ساختار با GraphStore ادعاها یکسان نیست. پیوند نامناسب نیز می‌تواند موضوع نامربوط وارد کند.

انتخاب architecture در rag_settings ذخیره و در سؤال بعدی اعمال می‌شود. retrieval_mode و use_vector_store مربوط به مسیر hybrid هستند و در سایر معماری‌ها همان معنا را ندارند. نبود مدل یا خطای لایه‌های مدل‌محور معمولاً به مسیر ساده‌تر BM25 برمی‌گردد. پیاده‌سازی این راهبردها به معنی اثبات سود آن‌ها برای این مجموعه نیست؛ مقایسه باید بازیابی، پاسخ، تأخیر و هزینه را با هم بسنجد.

Sources: compiler/rag_architectures.py | compiler/rag_settings.py | frontend/src/client/rag-architecture.ts

۱۶ | ذخیره‌سازی: JSON، SQLite و گراف

این پروژه برای شروع به یک پایگاه دادهٔ مرکزی و سنگین متکی نیست. بخش زیادی از وضعیت با فایل JSON و متن Markdown نگهداری می‌شود و بعضی داده‌های قابل جست‌وجو یا کش در SQLite قرار می‌گیرند. مزیت آن سادگی و قابل‌مشاهده بودن فایل‌هاست؛ هزینهٔ آن، محدودیت مقیاس، همزمانی و هماهنگ نگه‌داشتن چند فایل است.

ذخیره‌ساز	محتوا	نحوهٔ استفاده
Markdown	صفحه‌های موضوعی و ارجاع‌ها	خروجی قابل بازسازی و ورودی corpus ویکی.
JSON	تنظیمات، وضعیت، نشست و تاریخچه	بسیاری از سرویس‌های TypeScript مستقیم می‌خوانند.
کش LLM در SQLite	پاسخ‌های متن، تصویر، صوت و embedding	برای درخواست همسان استفادهٔ مجدد می‌شود.
VectorStore	بردار و فرادادهٔ passage	جست‌وجوی کسینوسی پایدار، بدون ANN.
GraphStore	گره و یالِ ادعاها و گروه‌ها	ورود و خروج ClaimGroup و اجرای انتشار روی ذخیره‌ساز.

شناسهٔ passage از مسیر، عنوان بخش و متن ساخته می‌شود تا تغییر محتوا به شناسهٔ تازه برسد. تابع همگام‌سازی بردار می‌تواند passageهای موجود را دوباره embedding نکند و نتیجهٔ جست‌وجو به corpus جاری محدود شود. use_vector_store در مسیر hybrid قابل تنظیم است، اما کامپایل عادی همهٔ ذخیره‌سازهای پژوهشی را خودکار پر نمی‌کند.

VectorStore بردارها را به شکل باینری و همراه norm نگه می‌دارد، ولی هنگام جست‌وجو همچنان اسکن خطی دارد. benchmark مخزن با بردار تصادفی، هزینهٔ زیرساخت را می‌سنجد؛ از آن نمی‌توان کیفیت معنایی بازیابی را نتیجه گرفت. نوشتن اتمی فایل در بک‌اند خطر فایل نیمه‌نوشته را کم می‌کند، اما به‌تنهایی معادل تراکنش مشترک میان همهٔ فایل‌ها نیست.

Sources: compiler/vector_store.py | compiler/graph_store.py | compiler/rag_engine.py | backend/src/lib/atomicWrite.ts

۱۷ | رابط کاربری: هر صفحه برای چه کاری است؟

مسیر فعلی	کار اصلی
/dashboard	مدیریت ورودی‌ها و پوشه‌ها، مشاهدهٔ وضعیت و آغاز پردازش.
/wiki	فهرست صفحه‌های تولیدشده، دسته‌بندی و ورود به صفحه.
/wiki/:slug	نمایش متن Markdown، ویرایش، دریافت و حذف صفحه.
/chat	پرسش از مجموعه و مشاهدهٔ پاسخ و ارجاع.
/resources	منابع و ارتباط آن‌ها با خروجی‌ها؛ در رابط فعلی محل برخی ابزارهای واردکردن.
/graph	نمای گراف تعاملی موضوع‌ها و پیوندهای ویکی.
/entities	نمای موجودیت‌های خوشه‌بندی‌شده و نام‌های جایگزین.
/pipelines	تاریخچه و جزئیات اجرای پردازش.
/pipeline-architecture	تنظیم گزینه‌های پردازش و پیش‌فرض‌ها.
/rag-architecture	انتخاب راهبرد بازیابی و تنظیم پاسخ.
/review-queue	رأی و یادداشت روی نامزدهای بازبینی پایلوت.
/logs · /usage · /analytics	ساختار و کیفیت داده، مصرف و رویدادهای عملیاتی.
/users · /company · /settings	مدل‌ها، زمینهٔ توصیفی مجموعه و مدیریت حساب‌ها.

صفحهٔ ویکی در کد فعلی بدنه را در pre با escaping نمایش می‌دهد؛ یک renderer کامل Markdown نیست. بنابراین نشانه‌های Markdown می‌توانند عیناً دیده شوند. گراف فعلی نیز با force-graph و دکمه‌های جست‌وجو، انتخاب و خروجی ساخته شده است. توضیح قدیمی «فقط فهرست گراف» یا «ویکی فقط دانلود دارد» وضعیت این نسخه را کامل توصیف نمی‌کند.

صفحهٔ عمومی آلمانی در / و انگلیسی در /en قرار دارد؛ /healthz پاسخ زنده‌بودن فرانت‌اند است. بخش‌های کاری پس از ورود قابل دسترسی‌اند. متن رابط و محتوای دانش دو لایهٔ متفاوت‌اند.

Sources: frontend/src/routes/simple.ts | frontend/src/routes/wiki.ts | frontend/src/views/wiki-page.ejs | frontend/src/client/graph.ts

۱۸ | فرانت‌اند، زبان‌ها و ظاهر

فرانت‌اند server-rendered است: Express دادهٔ لازم را از بک‌اند می‌گیرد و EJS قالب HTML می‌سازد. اسکریپت‌های TypeScript برای تعامل هر صفحه نوشته و با esbuild به bundle مستقل تبدیل می‌شوند. CSS Tailwind سبک‌ها را تولید می‌کند. این ساختار، یک برنامهٔ React یا SPA با router مرورگر نیست.

ترجمه چگونه کنترل می‌شود؟

رابط انگلیسی و آلمانی است. متن‌های رابط باید از t() و کلیدهای نام‌فضای صفحه استفاده کنند. فایل آلمانی با کلیدهای انگلیسی type-check می‌شود؛ ابزار check-i18n کلیدهای استفاده‌شده و جای‌نگهدارها را کنترل می‌کند. برای جمع‌ها و قالب تاریخ و عدد نیز helper وجود دارد. این PDF فارسی است، اما ساختن آن زبان فارسی را به رابط سایت اضافه نمی‌کند.

ترجیح زبان از کوکی و Accept-Language تعیین می‌شود و در صفحهٔ عمومی پیش‌فرض آلمانی و در برنامه پیش‌فرض انگلیسی است. proxy زبان را با X-Lang به بک‌اند می‌رساند تا پیام خطا نیز ترجمه شود. متن خود اسناد، پاسخ چت و دادهٔ آزاد الزاماً ترجمهٔ خودکار رابط نمی‌گیرند.

فونت و رنگ

Sora برای رابط و تیتر و Inter برای متن‌های بلند در پروژه تعریف شده‌اند. فایل‌های WOFF2 از بسته‌ها کپی و خودمیزبان ارائه می‌شوند. پالت فعلی از رنگ‌های گرم stone و amber استفاده می‌کند؛ برای محتوای تولیدشده indigo نیز دیده می‌شود. برخی رنگ‌های گراف در کد canvas ثابت‌اند و با تغییر Tailwind خودکار به‌روزرسانی نمی‌شوند.

```
cd frontend
npm run typecheck
# server types + client types + i18n checks
npm run build
# fonts + CSS + browser bundles + server
```

این گزارش فایل‌های views یا client را تغییر نداده است؛ فرمان‌های بالا شرح قرارداد توسعه‌اند و به معنی اجرای آزمون برنامه در این نوبت نیستند.

Sources: frontend/package.json | frontend/src/i18n/core.ts | frontend/src/index.ts | documentation/41-internationalization-and-fonts.md

۱۹ | بک‌اند و خانواده‌های اصلی API

بیشتر endpointها در backend/src/routes/index.ts ثبت می‌شوند. بخش‌های ساده در TypeScript روی فایل‌ها کار می‌کنند و بخش‌های تخصصی از bridge Python عبور می‌کنند. مسیر health و login پیش از دروازهٔ احراز هویت‌اند؛ بیشتر APIهای دیگر پس از requireAuth قرار می‌گیرند.

خانوادهٔ API	مسئولیت
/api/users · /api/auth/*	ورود، خروج، هویت جاری و مدیریت حساب.
/api/sources · /api/raw-files	فایل خام، بارگذاری، پوشه و ثبت مسیر منبع.
/api/docs/*	فهرست، خواندن، ویرایش و حذف خروجی.
/api/pipeline-runs · /api/build/*	شروع جریانی، توقف و تاریخچهٔ ساخت.
/api/chat/*	پرسش، وضعیت و مدیریت نشست‌های گفتگو.
/api/emails · /api/resources	منابع و اطلاعات ایمیل.
/api/knowledge-graph	گراف موضوعی و قواعد دستی رابطه.
/api/temporal-facts · /api/entity-graph	خوشهٔ موجودیت و دادهٔ زمانی.
/api/connectors/*	اتصال حساب، فهرست و واردکردن آیتم.
/api/review-queue	نامزد و ثبت تصمیم بازبینی.

پل CLI ورودی را از stdin می‌خواند و خروجی JSON می‌نویسد. خطای فرایند باید به وضعیت HTTP مناسب تبدیل شود. در مسیرهای جریانی، هر رویداد جداگانه flush می‌شود تا کاربر منتظر پایان کامل نماند. برای ساخت از SSE و برای کنترل مصرف منابع از حد همزمانی استفاده می‌شود.

کد فایل‌ها را با کنترل مسیر و محدودیت‌های لازم مدیریت می‌کند؛ منبع ثبت‌شده با symlinkهای فایل‌به‌فایل در data/raw آینه می‌شود. مسیر ثبت‌شده در Docker باید داخل container قابل دسترسی باشد. وجود یک مسیر روی لپ‌تاپ، خودبه‌خود آن را داخل container در دسترس نمی‌کند.

Sources: backend/src/routes/index.ts | backend/src/lib/pythonBridge.ts | backend/src/lib/sourcesRegistry.ts | compiler/cli.py

۲۰ | ورود، کاربران و مدل اعتماد تیمی

کاربران نقش admin یا user دارند. ثبت‌نام عمومی مدل اصلی نیست؛ مدیر حساب‌ها را می‌سازد. رمزها با bcryptjs هش می‌شوند. نشست‌ها توکن‌های تصادفی opaque هستند و در فایل ذخیره می‌شوند؛ JWT امضاشده نیستند. مدت پیش‌فرض نشست ۳۰ روز است و با حذف رکورد می‌توان آن را لغو کرد.

فرانت‌اند توکن را در کوکی HttpOnly نگه می‌دارد. در محیط HTTPS گزینهٔ Secure فعال است و Lax=SameSite استفاده می‌شود. مرورگر لازم نیست توکن را در JavaScript بخواند؛ proxy آن را برای بک‌اند به Bearer تبدیل می‌کند. محدودسازی تلاش ورود، شکست‌ها را بر اساس IP و نام کاربری دنبال می‌کند.

عملیات	قانون مهم
ساخت مدیر اولیه	فقط وقتی هنوز حسابی نیست؛ در production رمز اولیه الزامی است.
تغییر نقش یا رمز	نشست‌های کاربر لغو می‌شوند تا اختیار قبلی باقی نماند.
حذف کاربر	نشست‌ها نیز لغو می‌شوند؛ حذف خود و آخرین مدیر محدود شده است.
مدیریت کاربران	به requireAdmin نیاز دارد.
بیشتر کارهای دانش و فایل	برای عضو واردشدهٔ تیم مجاز است.

این آخرین ردیف برای فهم دامنهٔ پروژه مهم است. مدل فعلی «یک تیم مورداعتماد با نقش مدیریتی» است. مدیر تعیین می‌کند چه کسی وارد شود، اما بیشتر عملیات کامپایل، فایل و اتصال فقط به ورود نیاز دارند. جداسازی کامل دانش و اعتبارنامه به ازای هر کاربر یا سازمان، از این ساختار نتیجه نمی‌شود.

> **مرز بررسی**
>
> وجود middleware و هدرهای امنیتی قابل مشاهده است؛ این گزارش آزمون نفوذ، ارزیابی امنیتی سرویس زنده یا تأیید جداسازی چندمستأجری نیست. برای فهم پروژه باید مدل دسترسی واقعی کد را در نظر گرفت.

Sources: backend/src/lib/users.ts | backend/src/lib/sessions.ts | backend/src/lib/authMiddleware.ts | backend/src/routes/index.ts | frontend/src/routes/auth.ts

۲۱ | حریم خصوصی و مسیر خروج داده

pii_redaction یک پالایش محلی و عمدتاً regex است. سیاست پیش‌فرض، الگوهایی مانند شمارهٔ تأمین اجتماعی، کارت با بررسی Luhn، کلید محرمانه، تلفن و IPv4 را با جای‌نگهدار پایدار در همان فراخوانی عوض می‌کند. ایمیل و نام افراد به‌طور پیش‌فرض باقی می‌مانند تا تشخیص موجودیت مختل نشود. سیاست سخت‌گیرانه‌تر برای ایمیل نیز وجود دارد.

اتصال واقعی پالایش در این نسخه

پرچم redaction در main.py به‌طور پیش‌فرض روشن است و در extract_chunk_topics روی متن مورد استفاده در prompt استخراج اعمال می‌شود. متن اصلی در state محلی حفظ می‌شود. در کد ترکیب، chunk_blocks از entry.text اصلی ساخته می‌شوند و پارامتر مشابهِ پالایش سراسری دیده نمی‌شود. پس نباید ادعا کرد همهٔ داده‌ها پیش از تمام تماس‌های مدل پوشانده می‌شوند.

مسیر	آنچه باید بدانیم
استخراج متن	گذر پالایش در این مسیر صریحاً وصل شده است.
ترکیب، منتقد و چت	متن ورودی و پوشش پالایش باید جدا بررسی شود.
تصویر و صوت	دادهٔ رسانه می‌تواند به endpoint مدل فرستاده شود.
کش و وضعیت محلی	می‌توانند متن و پاسخ نگه دارند؛ حذف خروجی معادل حذف همهٔ داده نیست.
اتصال‌های بیرونی	اعتبارنامهٔ connector با Fernet رمز می‌شود؛ این قرارداد را نباید به همهٔ فایل‌های تنظیمات تعمیم داد.

NER اختیاری برای مکان‌ها قابل تزریق است، ولی مسیر پایه بدون آن کار می‌کند. regex نمی‌تواند همهٔ اطلاعات حساس بیان‌شده در زبان طبیعی را پیدا کند و ممکن است شناسهٔ معمولی را با secret اشتباه بگیرد. اعداد خوب روی fixtureهای محدود، پوشش همهٔ زبان‌ها و قالب‌ها را ثابت نمی‌کنند.

در تهیهٔ گزارش، فایل env.، رمزها، توکن‌های نشست و متن گفتگوهای شخصی خوانده یا بازنشر نشده‌اند. شرح حاضر ساختار و مرزهای کد را توضیح می‌دهد.

Sources: compiler/pii_redaction.py | compiler/synthesizer.py | compiler/main.py | compiler/connectors/credential_store.py

۲۲ | اتصال به Gmail، Drive و IMAP

اتصال‌ها یک قرارداد مشترک دارند: list_items خلاصهٔ آیتم‌های قابل واردکردن را نشان می‌دهد و fetch_item متن یک آیتم را دریافت می‌کند. connectors_service گردش کار حساب و واردکردن را هماهنگ می‌کند. دادهٔ دریافتی زیر data/raw/connectors قرار می‌گیرد و بعد مانند فایل‌های دیگر وارد کامپایل می‌شود.

اتصال	روش ورود	نوع آیتم
Gmail	OAuth2 با scope خواندن	پیام و متن قابل استخراج.
Drive Google	OAuth2 با scope خواندن	فایل و متن قابل دریافت توسط adapter.
IMAP	میزبان، پورت، mailbox و رمز برنامه	پیام صندوق انتخاب‌شده.

OAuth از PKCE و state تصادفی استفاده می‌کند. چون هر درخواست Python یک فرایند تازه است، verifier در حافظهٔ ماندگار یک سرور Python باقی نمی‌ماند؛ دادهٔ pending روی دیسک با مهلت محدود ذخیره می‌شود و هنگام callback مصرف می‌شود. refresh توکن هنگام فهرست‌گیری یا واردکردن انجام و نتیجه ذخیره می‌شود.

CredentialStore دادهٔ حساب را با Fernet و کلید CONNECTOR_SECRET_KEY رمز می‌کند. نبود کلید نباید به ذخیرهٔ بی‌رمز تبدیل شود. Gmail و Drive در این adapterها مجوز نوشتن، ارسال ایمیل یا اشتراک‌گذاری نمی‌خواهند. در IMAP، توصیهٔ پروژه استفاده از رمز برنامهٔ ارائه‌دهنده است، نه رمز معمول حساب.

این اتصال، همگام‌سازی دائمی نیست

الگوی اصلی، مرور و انتخاب آیتم و واردکردن آن است. وجود connector به‌تنهایی به معنی مانیتورینگ دائمی صندوق، به‌روزرسانی خودکار نسخهٔ واردشده یا حذف خروجی بعد از حذف راه‌دور نیست. مسیر رابط نیز نسبت به مستند قدیمی تغییر کرده و قابلیت‌های مربوط در Resources متمرکز شده‌اند؛ callbackهای OAuth مسیر مستقل دارند.

> **وضعیت آزمون**
>
> آزمون‌های adapterها با HTTP یا IMAP تزریق‌شده و پاسخ مصنوعی طراحی شده‌اند. آن‌ها رفتار کد را می‌سنجند؛ در این گزارش هیچ حساب واقعی متصل یا آزموده نشده است.

Sources: compiler/connectors/base.py | compiler/connectors/oauth2.py | compiler/connectors_service.py | frontend/src/client/resources.ts

۲۳ | PostgreSQL و SQLite به‌عنوان منبع دانش

در اتصال پایگاه داده، آیتم یک جدول است. کاربر جدول‌ها را می‌بیند و جدول انتخاب‌شده به متن Markdown تبدیل می‌شود. متنِ snapshot به‌عنوان فایل ورودی ذخیره می‌شود و مدل آن را در خط لولهٔ عادی می‌خواند. این مسیر، اجرای سؤال آزاد SQL توسط چت نیست و جای ابزار تحلیل کامل پایگاه داده را نمی‌گیرد.

ویژگی	PostgreSQL	SQLite
محل داده	سرور شبکه با host و database و schema	فایل محلی قابل دسترسی برای یک‌اند.
شناسهٔ آیتم	schema.table	نام جدول.
فهرست‌گیری	information_schema و شمارش ردیف‌ها	sqlite_master و PRAGMA.
دریافت محتوا	حداکثر ۲۰۰ ردیف	حداکثر ۲۰۰ ردیف.
اعتبارنامه	رمز در مخزن رمزشدهٔ connector	مسیر فایل؛ به شبکه نیاز ندارد.

Postgres نام جدول را با schema تأیید و شناسه‌ها را quote می‌کند؛ مقدارهای ورودی نیز به‌صورت پارامتری استفاده می‌شوند. کد عملیات SELECT و خواندن فراداده دارد. نقش خواندنی در خود پایگاه، مرز مستقلی برای دسترسی فراهم می‌کند؛ در دادهٔ نمونه نباید نقش مالک ساده‌شده را الگوی آمادهٔ تولید دانست.

SQLite نیز برای خواندن جدول و خروجی متنی پیاده شده است، اما factory پیش‌فرض sqlite3.connect معمولی است؛ از این کد نمی‌توان تضمین اتصال در حالت URI فقطخواندنی نتیجه گرفت. این تفاوت با «کد عملیات نوشتن اجرا نمی‌کند» باید روشن بماند.

دادهٔ نمونه و محدودیت snapshot

Compose توسعه PostgreSQL نمونه با جدول‌های departments، employees، projects و kb_articles دارد. در production این نمونه پیش‌فرض اجرا نمی‌شود. سقف ۲۰۰ ردیف یعنی خروجی نمایندهٔ کل جدول بزرگ نیست؛ ترتیب SELECT بدون BY ORDER نیز قرارداد نمونه‌گیری تحلیلیِ ثابت ایجاد نمی‌کند. تغییر دادهٔ مبدأ تا واردکردن و ساخت بعدی لزوماً در ویکی دیده نمی‌شود.

Sources: compiler/connectors/postgres_db.py | compiler/connectors/sqlite_db.py | compiler/connectors_service.py | docker/postgres/init.sql

۲۴ | غنی‌سازی وب و مرز شواهد بیرونی

web_search در مرحلهٔ ترکیب می‌تواند برای موضوع‌های نیازمند بازسازی، نتیجهٔ اینترنت را به قطعه‌های موجود اضافه کند. پیش‌فرض خاموش است. با web-search-- یا تنظیم متناظر فعال می‌شود و نتیجه‌ها source_type برابر web می‌گیرند. URL به‌عنوان منبع در جدول ارجاع ثبت می‌شود و اعتماد اولیهٔ آن low است.

برای جلوگیری از صدها درخواست در یک مجموعهٔ بزرگ، تعداد نتیجه به ازای موضوع و تعداد موضوع در هر اجرا سقف دارد؛ پیش‌فرض‌های مستندشده سه نتیجه و هشت موضوع‌اند. ساخت افزایشی بدون تغییر محلی ممکن است هیچ موضوعی برای جست‌وجو نداشته باشد. روشن بودن گزینه به معنی تازه‌سازی همهٔ اطلاعات وب در هر اجرا نیست.

ارائه‌دهنده در کد	ویژگی مهم
DuckDuckGo	از Answer Instant استفاده می‌کند؛ پوشش آن معادل جست‌وجوی کامل وب نیست.
SerpAPI	به کلید تنظیم‌شده نیاز دارد.
Bing	adapter و تنظیم کلید دارد؛ دسترس‌پذیری سرویس در این بررسی تأیید نشده است.

اشتباه پیکربندی می‌تواند غنی‌سازی آن اجرا را متوقف کند؛ خطای یک query معمولاً همان موضوع را رد می‌کند تا کل ساخت به دلیل یک جست‌وجو از کار نیفتد. این ماژول جدا از connector حساب است: یک query عمومی می‌زند و از فهرست فایل‌های یک حساب وارد نمی‌کند.

> **اثر پژوهشی**
>
> با ورود شاهد وب، خروجی فقط تابع فایل‌های محلی نیست. زمان، query، ارائه‌دهنده و نتیجهٔ دریافت‌شده برای بازتولیدپذیری اهمیت پیدا می‌کنند. همچنین دادهٔ بیرونی مرتبط می‌تواند با شاهد اصلی تعارض داشته باشد و نباید بی‌نام با آن ادغام شود.

این PDF از جست‌وجوی وب برای بازنویسی اطلاعات پروژه استفاده نکرده است. توضیح این قابلیت از ماژول و قراردادهای موجود در مخزن می‌آید؛ وضعیت فعلی سرویس‌های ارائه‌دهنده موضوع ارزیابی جداست.

Sources: compiler/web_search.py | compiler/main.py | documentation/37-web-search-enrichment.md

۲۵ | تنظیمات مدل‌ها، پردازش و هزینه

تنظیمات LLM می‌تواند چند profile داشته باشد: نام نمایشی، provider، URL base، مدل، کلید و پارامترهای نمونه‌گیری. این profileها به چهار purpose نسبت داده می‌شوند. بک‌اند هنگام اجرای Python، متغیرهای محیطی متناظر را می‌سازد و LLMClient.for_purpose آن‌ها را می‌خواند.

purpose	کار مورد نظر
default	کارهای عمومی مانند استخراج و پیونددهی.
thinking	ترکیب صفحه که به یکپارچه‌کردن چند منبع نیاز دارد.
chat	پاسخ تعاملی به سؤال کاربر.
embedding	بردارسازی برای بازیابی معنایی.

نام profile یا provider تضمین نمی‌کند endpoint همهٔ قابلیت‌ها را دارد. یک سرور متن ممکن است تصویر، صوت یا embedding را نپذیرد. نام مدل‌های پیش‌فرض در کد، وضعیت امروز کاتالوگ ارائه‌دهنده یا دسترسی حساب را اثبات نمی‌کند. این گزارش آن‌ها را پیشنهاد خرید یا انتخاب مدل تلقی نمی‌کند.

سه دسته تنظیم جدا

pipeline_settings پیش‌فرض‌های ساخت، مانند منتقد، اصلاحات، پوشاندن PII و غنی‌سازی وب را نگه می‌دارد. rag_settings معماری، top_k، پارامترهای BM25، store vector و حالت پاسخ را کنترل می‌کند. company_settings اطلاعات توصیفی سازمان یا مجموعه را ذخیره می‌کند؛ طبق کد فعلی هنوز به prompt کامپایلر وصل نشده است.

مصرف توکن به تفکیک مرحله و مدل ثبت می‌شود. برآورد هزینه از جدول قیمت داخلی و مصرف گزارش‌شده به دست می‌آید و نباید با صورت‌حساب قطعی سرویس یکی دانسته شود. کش و ساخت افزایشی می‌توانند فراخوانی را کم کنند؛ منتقد چندنمونه‌ای، HyDE، Fusion و rerank ممکن است فراخوانی بیشتری بسازند.

> **نکتهٔ تنظیم عملی**
>
> تغییر معماری RAG برای سؤال بعدی خوانده می‌شود. اما تغییر مدل ساخت، اصلاحات یا منتقد لزوماً همهٔ صفحه‌های قبلی را بازسازی نمی‌کند؛ سیاست invalidation و force-- باید آگاهانه انتخاب شود.

Sources: backend/src/lib/llmSettings.ts | backend/src/lib/pipelineSettings.ts | backend/src/lib/companySettings.ts | compiler/rag_settings.py | compiler/llm_client.py

۲۶ | مشاهده‌پذیری: اجرا، گراف، آمار و لاگ

ساخت ویکی عملیاتی چندمرحله‌ای است. pipeline_tracker برای هر اجرا وضعیت، زمان، گزینه‌ها، پیشرفت مرحله و خطا را ثبت می‌کند. در رابط Pipelines می‌توان تفاوت «در حال اجرا»، «موفق»، «خطادار» و «متوقف‌شده» را دید. دادهٔ اخیر پردازش‌شده کمک می‌کند یک مرحلهٔ طولانی صرفاً بی‌حرکت به نظر نرسد.

پل بک‌اند خروجی Rich را برای نمایش پاک‌سازی می‌کند و رویدادهای SSE می‌فرستد. قطع مرورگر با پایان کار Python یکسان نیست؛ به همین دلیل شناسهٔ اجرای جاری و رکورد اجرای رهاشده اهمیت دارد. هنگام راه‌اندازی، رکوردهای درحال‌اجرایِ بی‌فرایند می‌توانند با وضعیت خطا تطبیق داده شوند.

نما یا سرویس	چه چیزی نشان می‌دهد؟	چه چیزی را اثبات نمی‌کند؟
Analytics	تعداد، برچسب‌ها و پیوند خراب	درستی علمی متن.
Graph	رابطهٔ موضوع‌ها و hubها	تأیید یا رابطهٔ علّی ادعاها.
Usage	مصرف ثبت‌شده و برآورد هزینه	صورتحساب نهایی ارائه‌دهنده.
Logs	ورود، تغییر فایل، اتصال و خطای برنامه	ثبت بی‌نقص همهٔ رخدادهای بیرونی.
API Attention	فایل پردازش‌نشده، موضوع یتیم، بن‌بست، پیوند خراب و یافتهٔ بازبین	حل خودکار مشکل یا اعتبارسنجی معنایی کامل.

در attentionEngine، موضوع بی‌پشتوانه به معنی عنوانی است که در وضعیت استخراج کنونی منبع متناظر ندارد؛ این شاخص ساختاری را نباید معادل تشخیص قطعی توهم دانست. موضوع یتیم نیز ممکن است محتوای درست داشته باشد، اما مسیر مرور به آن ضعیف باشد.

connectorActivity جزئیات اتصال، فهرست‌گیری و واردکردن، زمان و موفقیت یا شکست را جدا ثبت می‌کند. این داده برای تشخیص مشکل شبکه، اعتبارنامه یا دسترسی فایل مفید است. تحلیل خطا باید از مرحله و شناسهٔ اجرا شروع شود، نه از تکرار بی‌هدف force--.

Sources: compiler/pipeline_tracker.py | backend/src/lib/pipelineRuns.ts | backend/src/lib/attentionEngine.ts | backend/src/lib/connectorActivity.ts

۲۷ | راه‌اندازی محلی بدون Docker

برای فهم اجرای پروژه، باید محیط Python و دو محیط Node را جدا آماده کرد. فرمان‌های زیر دستورهای نمونه بر اساس packageهای فعلی‌اند؛ در این گزارش سرویس برنامه، کامپایل مدل یا اتصال بیرونی اجرا نشده است. فایل env.example. قرارداد متغیرها را نشان می‌دهد؛ مقدار محرمانه باید در تنظیمات محلی خود کاربر بماند.

```
cp .env.example .env

cd compiler
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python main.py --force
```

کامپایل به endpoint سازگار و کلید یا مقدار تنظیم‌شدهٔ لازم نیاز دارد. برای مسیر محلی نیز require_llm انتظار کلید غیرخالی دارد، هرچند سرور محلی ممکن است آن را صرفاً به‌عنوان قرارداد بپذیرد. اجرای force-- روی مجموعهٔ بزرگ می‌تواند زمان و هزینه داشته باشد.

```
# Terminal A: backend
cd backend
npm install
export PYTHON_BIN=/absolute/path/to/wiki/compiler/.venv/bin/python
npm run dev

# Terminal B: frontend
cd frontend
npm install
npm run build
npm run dev:server
```

backend/package.json فرمان dev دارد؛ برخی راهنماهای قدیمی به dev:server برای بک‌اند اشاره می‌کنند. برای فرانت‌اند dev:server وجود دارد. پس از تغییر CSS یا کد مرورگر، از watchهای dev:css و dev:client یا build مجدد استفاده می‌شود. داشتن Python روی PATH کافی نیست اگر کتابخانه‌ها در venv دیگری نصب شده باشند؛ PYTHON_BIN باید به مفسر درست اشاره کند.

نشانی محلی	معنای بررسی
http://localhost:3000/healthz	زنده‌بودن فرانت‌اند.
http://localhost:8000/api/health	زنده‌بودن API بک‌اند.
http://localhost:3000/login	ورود و شروع استفاده از برنامه.

پاسخ health به‌تنهایی موفقیت کامپایل یا کیفیت پاسخ چت را تأیید نمی‌کند. مسیر build_wiki.sh نیز برای آماده‌سازی و ساخت وجود دارد، ولی نباید فرمان قدیمی npm start در wiki-app را مسیر فعلی رابط تصور کرد.

Sources: frontend/package.json | backend/package.json | compiler/requirements.txt | build_wiki.sh | documentation/02-getting-started.md

۲۸ | Docker توسعه و مدل محلی

Compose توسعه دو سرویس برنامه و PostgreSQL نمونه را هماهنگ می‌کند؛ profile اختیاری local-llm نیز دارد. تصویر بک‌اند باید هم Node و هم Python و compiler را حمل کند، چون subprocess داخل همان container اجرا می‌شود. تصویر فرانت‌اند تنها مسئول رابط است و از طریق API به داده می‌رسد.

```
docker compose up --build

# Optional local model service
docker compose --profile local-llm up --build
```

داده و خروجی با mount bind حفظ می‌شوند. در حالت توسعه، compiler می‌تواند به‌صورت پوشهٔ زنده mount شود؛ ولی تغییر TypeScript داخل تصویر لزوماً بدون build تازه اعمال نمی‌شود. نام backend داخل شبکهٔ Compose برای فرانت‌اند قابل حل است و نام localhost از داخل container به همان container اشاره می‌کند.

مدل محلی واقعاً چیست؟

entrypoint مدل GGUF را در صورت نبودن از مخزن تنظیم‌شده می‌گیرد و python -m llama_cpp.server را اجرا می‌کند. فایل مدل در volume ماندگار می‌ماند تا هر راه‌اندازی دوباره دانلود نشود. کد فعلی از hf download استفاده می‌کند؛ بعضی متن‌های قدیمی هنوز huggingface-cli را نوشته‌اند. این سرویس Ollama نیست.

URL base کامپایلر باید به سرویس محلی و alias مدل اشاره کند. نام repository و فایل پیش‌فرض در مستندات به‌صراحت تأییدنشده معرفی شده است؛ برای اجرا باید با فایل واقعیِ در دسترس تطبیق داده شود. وجود profile به معنی آماده‌بودن وزن مدل، مجوز دسترسی یا توان سخت‌افزار نیست.

> **مرز محلی‌بودن**
>
> این container برای completion chat تنظیم شده است. embedding، رونویسی صوت و توانایی تصویر ممکن است به endpoint یا مدل دیگری نیاز داشته باشند. «مدل متن محلی» لزوماً تمام خط لوله را بدون سرویس بیرونی کامل نمی‌کند.

خطای endpoint باید از خطای parser فایل یا مرحلهٔ ترکیب جدا تشخیص داده شود. تنزل BM25 یا پیوست بدون رونویسی، ممکن است برنامه را فعال نگه دارد ولی محتوای کمتری برای پاسخ فراهم کند.

Sources: docker-compose.yml | backend/Dockerfile | frontend/Dockerfile | docker/local-llm/entrypoint.sh | documentation/33-docker-and-local-llm.md

۲۹ | استقرار و نگهداری محیط production

docker-compose.prod.yml برای میزبان کوچک تنظیم شده است. backend تنها در شبکهٔ داخلی پورت ۸۰۰۰ را expose می‌کند؛ فرانت‌اند به‌طور پیش‌فرض روی پورت میزبان ۳۰۰۵ نگاشت می‌شود. ورودی عمومی در طرح مستندشده از Cloudflare به فرانت‌اند می‌رسد. profile اختیاری Caddy برای سناریوی TLS مستقیم نیز وجود دارد.

```
cp .env.example .env
# configure required values locally
./deploy/deploy.sh

# update workflow
./deploy/deploy.sh --pull
```

production رمز ADMIN_PASSWORD را الزامی می‌کند. نمونهٔ Postgres و local-llm در این فایل پیش‌فرض نیستند؛ hint رمز و پایگاه نمونه نیز مخفی می‌شوند. کوکی Secure، محدودیت حافظه، سقف Node heap، محدودیت subprocess و گردش لاگ بخشی از تنظیمات‌اند. سقف‌های پیش‌فرض ۷۶۸ مگابایت برای بک‌اند و ۱۶۰ مگابایت برای فرانت‌اند هستند؛ این‌ها تنظیم‌اند، نه مصرف اندازه‌گیری‌شده در این بررسی.

چه چیزی باید حفظ شود؟

data شامل فایل خام، وضعیت، کاربران، نشست، گفتگو و تنظیمات است. wiki-app/docs و static/media نیز بخشی از وضعیت قابل استفاده‌اند. کلید رمز connector و تنظیمات لازم برای بازگشایی اعتبارنامه باید از مسیر امن پشتیبان‌گیری شوند؛ داشتن فایل رمزشده بدون کلید برای بازیابی کافی نیست. کش مدل بازسازی‌پذیر است اما حذف آن می‌تواند هزینه ایجاد کند.

اسکریپت deploy مالکیت مسیرها را برای کاربر container تنظیم می‌کند. volume باید قابل نوشتن باشد؛ خطای EACCES معمولاً به همین مرز مربوط است. healthcheck ترتیب شروع سرویس‌ها را کمک می‌کند، ولی موفقیت semantic ساخت و چت را نشان نمی‌دهد.

> **برداشت از مستند استقرار**
>
> این صفحه توصیف تنظیمات مخزن است و وضعیت زندهٔ DNS، Tunnel، گواهی و سرور را تأیید نمی‌کند. بخش‌های قدیمی دربارهٔ Pages GitHub مسیر deployment فعلی را نشان نمی‌دهند؛ workflow موجود روی کنترل PR تمرکز دارد.

Sources: docker-compose.prod.yml | deploy/deploy.sh | deploy/Caddyfile | .github/workflows/pr-checks.yml | documentation/40-production-deployment.md

۳۰ | آزمون‌ها و ارزیابی کیفیت

پروژه چند لایهٔ آزمون دارد. pytest منطق Python، parserها، خط لوله با مدل مصنوعی و الگوریتم‌ها را بررسی می‌کند. Vitest برای سرویس‌های TypeScript بک‌اند است. typecheck و ESLint خطاهای نوع و برخی الگوهای کد را پیدا می‌کنند. کنترل i18n بخشی از بررسی فرانت‌اند است. CI موجود چهار job برای compiler، backend، frontend و Docker دارد.

```
# Start at the repository root
cd compiler
python -m pytest -q
ruff check .

# Backend
cd ../backend
npm test
npm run typecheck

# Frontend
cd ../frontend
npm run typecheck
npm run build
```

ابزار ارزیابی	پرسش مورد سنجش
trust_propagation_eval	آیا امتیاز ادعای خوب بالاتر از نامعتبر قرار می‌گیرد؟
retrieval_eval	Recall و nDCG برای مرتبط‌بودن بازیابی.
entity_resolution_eval	درستی و پوشش ادغام نام‌ها.
temporal_model_eval	انتخاب ادعاهای جاری با رابطهٔ زمانی.
extraction_critic_eval	دقت و پوشش علامت‌گذاری جملهٔ بی‌پشتوانه.
faithfulness_eval	وفاداری پاسخ به passageهای ارائه‌شده.
pii_redaction_eval	رفتار detectorها روی fixtureهای برچسب‌خورده.
scalability_benchmark	هزینهٔ زیرساخت در اندازه‌های مختلف.

مدل مصنوعی برای سنجیدن prompt، parse، cache و fallback مفید است، اما توان فهم مدل واقعی را نمی‌سنجد. آزمون connector با پاسخ ساختگی نیز زنده‌بودن حساب یا سازگاری آخرین نسخهٔ API را اثبات نمی‌کند. این گزارش آزمون‌های برنامه را اجرا نکرده؛ شمار فایل‌های آزمون صرفاً موجودی مخزن است. برای خود PDF کنترل متن، صفحه‌بندی و رندر جدا انجام شده است.

Sources: compiler/tests/ | backend/package.json | frontend/package.json | .github/workflows/pr-checks.yml

۳۱ | پروتکل پژوهش و مطالعهٔ کاربری

user_study.py و مستند ۳۲ ابزار و طرح یک مطالعه‌اند. پرسش آن این است که آیا ویکی و چت نسبت به جست‌وجوی مستقیم در همان یادداشت‌ها و ایمیل‌ها، یافتن پاسخ را سریع‌تر و دقیق‌تر می‌کند. طبق دستور پروژه و مستندات، مطالعه هنوز اجرا نشده و نتیجهٔ واقعی وجود ندارد.

جزء طرح	شرح
طرح درون‌فردی	هر شرکت‌کننده هر دو وضعیت را تجربه می‌کند.
وضعیت A	استفاده از ویکی و چت.
وضعیت B	جست‌وجوی مستقیم در منابع خام با ابزار معمول.
توازن ترتیب	برای همه، یک وضعیت همیشه اول نیست؛ ترتیب وظیفه‌ها هم جابه‌جا می‌شود.
وظیفه‌ها	هشت سؤال مبتنی بر دادهٔ نمونه و شواهد برچسب‌خورده.
خروجی ثبت‌شده	مدت، درستی پاسخ و اعتمادبه‌نفس گزارش‌شده.

توازن ترتیب برای کاهش اثر یادگیری اهمیت دارد. اگر همه اول از ابزار A و بعد B استفاده کنند، وضعیت دوم ممکن است فقط به دلیل آشنایی با سؤال سریع‌تر شود. seed مشخص اجازه می‌دهد تخصیص ترتیب دوباره ساخته شود. دادهٔ مصنوعیِ آزمون فقط برای بررسی محاسبهٔ آمار استفاده می‌شود و نباید نتیجهٔ پژوهش نامیده شود.

از ارزیابی الگوریتم تا سود واقعی

Recall بهتر الزاماً تجربهٔ بهتر نمی‌سازد؛ کاربر ممکن است نتواند ارجاع را باز کند، پاسخ طولانی را بخواند یا تعارض را بفهمد. به همین دلیل اندازه‌گیری retrieval و مطالعهٔ کاربر مکمل‌اند. همچنین خروجی یک داور مدل، جای رأی مستقل انسان را به‌طور خودکار نمی‌گیرد.

> **ادعاهای قابل دفاع در این مرحله**
>
> می‌توان گفت پروتکل، ابزار ثبت و ارزیاب‌های فنی در مخزن وجود دارند. نمی‌توان گفت ویسنسباو زمان جست‌وجو را به مقدار مشخص کاهش داده یا برای کاربران به‌طور معنادار بهتر است. چنین نتیجه‌ای به اجرای واقعی و تحلیل داده نیاز دارد.

Sources: compiler/user_study.py | compiler/tests/test_user_study.py | documentation/32-user-study-protocol.md | AGENTS.md

۳۲ | محدودیت‌ها و اختلاف مستند با کد

موضوع	شرح دقیق نسخهٔ بررسی‌شده
معماری رابط	Express/EJS فعلی است؛ Docusaurus و React در بعضی متن‌ها تاریخی‌اند.
نمای ویکی	مسیر صفحه و edit وجود دارد؛ بدنه در pre نمایش داده می‌شود.
گراف رابط	force-graph تعاملی وجود دارد؛ شرح قدیمی گراف فقط‌فهرست کامل نیست.
صف بازبینی	به پایلوت ادعاها وابسته است؛ پوشش خودکار همهٔ ورودی‌ها نیست.
زمان	API دارد؛ منبع زمانی هنوز پایلوت است و دو محور زمان کامل جدا نشده‌اند.
پوشاندن PII	در استخراج وصل است؛ پوشش همهٔ تماس‌های مدل نتیجه نمی‌شود.
Company	ذخیرهٔ توصیف دارد؛ هنوز زمینهٔ prompt کامپایلر نیست.
ذخیره‌سازها	مسیر استفاده دارند؛ main همهٔ گراف‌ها و بردارها را خودکار آماده نمی‌کند.
مدل محلی	سرویس واقعیِ اجرای مدل دارد؛ نام وزن پیش‌فرض در مستند تأییدنشده است.
دسترسی	یک تیم مورداعتماد؛ جداسازی کامل چند سازمان اثبات نشده است.
استقرار	Compose تولید و deploy.sh موجود است؛ CI فعلی کنترل PR است.

اختلاف مستندات طبیعیِ تغییر معماری است، اما بر توضیح «کل پروژه» اثر مستقیم دارد. در این گزارش، برای مسیرها، فرمان‌ها، نمایش ویکی، proxy و SQLite به کد فعلی اولویت داده شده است. متون قدیمی همچنان برای نیت طراحی مفیدند، ولی نباید بدون تطبیق به‌عنوان رفتار قطعی بازگو شوند.

پژوهش نیز چند شکاف مستقل دارد: دادهٔ کوچک و داستانی، برچسب‌گذاری محدود، کیفیت داوری مدل واقعی، وابستگی بین گروه ادعاها و اجرای مطالعهٔ انسانی. هیچ‌کدام با زیاد بودن تعداد صفحه یا آزمون جایگزین نمی‌شوند.

Sources: frontend/src/routes/wiki.ts | frontend/src/client/graph.ts | compiler/cli.py | backend/src/lib/companySettings.ts | compiler/synthesizer.py

۳۳ | یک مثال سرتاسری برای فهم پروژه

فرض آموزشی: یک تیم سه مدرک دربارهٔ «فاصلهٔ نمونه‌برداری حسگر» دارد: یادداشت شروع پروژه مقدار ساعتی را می‌گوید، مشخصات جدید ۱۵ دقیقه را ذکر می‌کند و ایمیل مهندس توضیح می‌دهد نسخهٔ قبلی اصلاح شده است. این یک سناریوی فهم معماری است؛ در تهیهٔ گزارش فایلی ایجاد یا کامپایل نشده است.

- ورودی: سه فایل با مسیر و نوع مستقل در data/raw قرار می‌گیرند یا وارد می‌شوند. اصل آن‌ها حفظ می‌شود.
- کشف و استخراج: کامپایلر قطعه‌ها را می‌سازد و موضوع محصول، نمونه‌برداری و باتری را استخراج می‌کند. نام مهندس به‌عنوان موجودیت ثبت می‌شود.
- ترکیب: قطعه‌های هر موضوع کنار هم قرار می‌گیرند. صفحهٔ محصول باید تفاوت قدیم و جدید را توصیف کند و جدول منابع از هر سه مدرک ساخته شود.
- پیوند: عنوان باتری و شبکه به صفحه‌های موجود وصل می‌شوند؛ index و فهرست مطالب به‌روزرسانی می‌شوند.
- مرور: کاربر صفحه و Resources را بررسی می‌کند تا بداند کدام مدرک در کدام خروجی استفاده شده است.
- سؤال: چت برای «فاصلهٔ فعلی چیست؟» بخش‌های مرتبط را رتبه‌بندی می‌کند و پاسخ را با ارجاع ارائه می‌دهد.

اگر کاربر بعداً منبع را تغییر دهد، هش جدید باعث پردازش مجدد می‌شود. اگر فقط متن نهایی ویکی را دستی عوض کند، این اصلاح ممکن است در ساخت بعدی بازنویسی شود. برای رفع خطای پیوند می‌توان قاعدهٔ پایدار ثبت کرد؛ برای اصلاح prompt، حلقهٔ بازبینی موجود است اما نامزدهای آن هنوز پایلوت‌اند.

> **بخشی که نباید فرض کرد**
>
> خط لولهٔ عادی لزوماً از این سه سند به‌طور خودکار یک گراف ادعا با supersedes نمی‌سازد. مدل زمانی و انتشار اعتماد وقتی آن ساختار را داشته باشند کار می‌کنند. این مرز بین ترکیب متنیِ موجود و استنتاج ساختاریِ پژوهشی است.

Sources: compiler/main.py | compiler/synthesizer.py | compiler/linker.py | compiler/rag_engine.py | compiler/temporal_model.py

۳۴ | نگهداری و عیب‌یابی مرحله‌به‌مرحله

نشانه	از کجا شروع کنیم؟	دلیل محتمل
صفحهٔ عمومی باز است، برنامه خطا دارد	فرانت‌اند، BACKEND_API_URL و health بک‌اند	SSR به API یا نشست دسترسی ندارد.
فایل دیده نمی‌شود	پسوند شناخته‌شده، mount و symlink	فایل در دید فرایند یا مسیر ثبت‌شده نیست.
فایل هست، موضوع ندارد	state و مرحلهٔ Extraction	متن خالی، parser ناقص یا استخراج نامناسب.
فقط پیوست نمایش داده می‌شود	handler رسانه و وابستگی parser	قالب opaque یا شکست استخراج محتوا.
تنظیم جدید اثر نکرده	topics dirty و force--	محتوای فایل تغییر نکرده و مرحله رد شده است.
چت پاسخ اقتباسی می‌دهد	حالت پاسخ و اتصال مدل	انتخاب extractive یا شکست تولید مدل.
پاسخ نامربوط است	passage بازیابی‌شده و scope و architecture	مسئلهٔ retrieval پیش از generation.
ساخت متوقف به نظر می‌رسد	مرحله، آخرین پیشرفت و فرایند Python	تأخیر مدل، صف، محدودیت حافظه یا خطای فرایند.
خطای نوشتن در container	مالکیت volume و مجوز مسیر	uid اجرا به پوشهٔ داده دسترسی ندارد.

بهتر است یک ورودی کوچک و یک سؤال مشخص را تا آخر مسیر دنبال کنیم: فایل ← chunk ← topic ← draft ← خروجی ← passage ← پاسخ. این روش محل گم‌شدن اطلاعات را آشکار می‌کند. اجرای مکرر force-- بدون یافتن مرحلهٔ خطا، هزینه و نویز بیشتری می‌سازد.

در نگهداری دانش، پیوند شکسته، صفحهٔ یتیم و عنوان بدون منبع را بررسی کنید. اگر ویرایش تولیدشده باید پایدار بماند، پیش از کامپایل بعدی دربارهٔ محل ثبت اصلاح تصمیم بگیرید. حذف داده، رسانه، کش و تاریخچه عملیات متفاوت‌اند و دامنهٔ حذف باید از مسیرهای کد فهمیده شود.

Sources: documentation/16-troubleshooting.md | backend/src/lib/attentionEngine.ts | backend/src/lib/pythonBridge.ts | compiler/main.py

۳۵ | نقشهٔ ماژول‌ها برای ادامهٔ توسعه

اگر می‌خواهید این بخش را بفهمید یا تغییر دهید	نقطهٔ شروع
ترتیب مرحله‌ها و گزینه‌های CLI	compiler/main.py
کشف فایل، استخراج و ترکیب	compiler/synthesizer.py
ساخت chunk از رسانه و ایمیل	compiler/media_ingest.py · email_ingest.py
مدل، cache، retry و purpose	compiler/llm_client.py
پیوند و alias و قواعد دستی	compiler/linker.py · mechanical_linker.py
منابع و اعتماد اولیه	compiler/trust.py · resources_engine.py
ساخت corpus و پاسخ	compiler/rag_engine.py · rag_types.py
فرمول‌های retrieval	compiler/hybrid_retrieval.py
معماری‌های متفاوت retrieval	compiler/rag_architectures.py
ادعا و اعتماد رابطه‌ای	compiler/trust_propagation.py · graph_store.py
تشخیص نام و نمای موجودیت	compiler/entity_resolution.py · entity_graph.py
زمان و اصلاح انسان	compiler/temporal_model.py · active_learning.py
تبادل دادهٔ Node و Python	compiler/cli.py · backend/src/lib/pythonBridge.ts
رفتار endpointها	backend/src/routes/index.ts
رابط صفحه و تعامل مرورگر	frontend/src/views/ · frontend/src/client/
زبان و کلیدهای متن	frontend/src/i18n/ · scripts/check-i18n.mjs
استقرار و تصویرها	docker-compose.prod.yml · deploy/ · Dockerfile

وابستگی‌ها را یک‌جهته نگه دارید: نوع‌های مشترک RAG در rag_types هستند تا ماژول راهبردها مجبور به import موتور dispatchکنندهٔ خود نباشد. منطق خالص نیز از endpoint جداست تا بدون سرور یا سرویس بیرونی آزموده شود. این الگوی طراحی در engineها و connectorهای تزریق‌پذیر تکرار شده است.

برای تغییر رفتار داده، آزمونی بنویسید که قرارداد واقعی را بسنجد؛ برای تغییر رابط، typecheck و i18n الزامی‌اند. دادهٔ خام و تنظیمات محرمانه نباید به fixture عمومی یا متن گزارش منتقل شوند.

Sources: compiler/rag_types.py | compiler/cli.py | backend/src/lib/pythonBridge.ts | frontend/package.json | AGENTS.md

۳۶ | واژه‌نامهٔ کاربردی

اصطلاح	معنی در همین پروژه
Compiler / کامپایلر	زنجیرهٔ تبدیل فایل خام به صفحهٔ ویکی؛ کامپایلر زبان برنامه‌نویسی نیست.
source Raw / منبع خام	فایل انسانی یا snapshot واردشده که شاهد اولیه محسوب می‌شود.
Chunk / قطعه	بخش متن ورودی برای استخراج و ترکیب.
Topic / موضوع	محوری که قطعه‌ها بر اساس آن در یک صفحه جمع می‌شوند.
Entity / موجودیت	شخص، سازمان، محصول یا چیز دارای هویت.
Concept / مفهوم	ایده یا اصطلاح انتزاعی استخراج‌شده از متن.
Synthesis / ترکیب	نوشتن یک صفحه از چند شاهد مرتبط.
Slug	نام نرمال‌شده برای شناسه، مسیر و فایل.
matter Front	فرادادهٔ YAML ابتدای Markdown، مانند عنوان و برچسب.
MOC	فهرست یا نقشهٔ محتوا در index.md.
Passage	بخش کوچک‌تر یک سند که واحد بازیابی چت است.
BM25	امتیازدهی واژگانی با وزن واژه و تنظیم طول متن.
Embedding	بردار عددی برای مقایسهٔ معنایی متن‌ها.
RRF	ادغام فهرست‌ها با موقعیت رتبه.
Reranker	مدلی یا روشی برای مرتب‌کردن دوبارهٔ نامزدهای بازیابی.
Grounding	پشتیبانی‌شدن ادعای خروجی توسط شواهد داده‌شده.
Faithfulness	وفاداری پاسخ به متن شواهد؛ جدا از حقیقت بیرونی.
Prior	اعتماد اولیه پیش از اثر روابط ادعاها.
Supersedes	رابطهٔ جهت‌دار جایگزینی ادعای قدیمی با جدید.
SSR / SSE	ارسال رویداد جریانی از سرور / تولید HTML در سرور.
Snapshot	کپی وضعیت یک منبع در زمان واردکردن؛ اتصال زندهٔ دائمی نیست.
Ablation	آزمایش حذف یک مؤلفه برای فهم سهم آن در نتیجه.

Sources: documentation/05-compiler-pipeline.md | documentation/25-hybrid-retrieval.md | documentation/22-trust-propagation-algorithm.md

۳۷ | منابع، مبنای گزارش و مسیر مطالعه

این گزارش بر خواندن مخزن محلی تکیه دارد. فایل‌های کد، مرجع رفتار اجرایی و راهنماها مرجع نیت و روش طراحی بوده‌اند. منابع هر صفحه پایین همان صفحه آمده است؛ این فهرست، مسیرهای اصلی برای ادامهٔ مطالعه را گروه‌بندی می‌کند. مسیرها نسبت به ریشهٔ پروژه هستند.

موضوع	منابع منتخب
هویت و دامنه	AGENTS.md · README.md · documentation/18-sample-domain.md
ساختار و خط لوله	documentation/03-architecture.md · 05-compiler-pipeline.md · 06-extraction-and-synthesis.md
پیوند و وضعیت	documentation/07-linking-moc-and-pages.md · 10-data-layout-and-state.md
رسانه و منابع	documentation/19-multimedia-email-and-trust.md · 20-email-resources-and-chat-engines.md
اعتماد و منتقد	documentation/21-trust-eval-dataset.md · 22-trust-propagation-algorithm.md · 23-trust-propagation-evaluation.md · 24-extraction-critic.md
بازیابی و موجودیت	documentation/25-hybrid-retrieval.md · 26-entity-resolution.md · 27-temporal-modeling.md
بازبینی و حریم خصوصی	documentation/28-faithfulness-evaluation.md · 29-active-learning.md · 30-pii-redaction.md · 35-review-queue-ui.md
ذخیره‌سازی و مطالعه	documentation/31-vector-graph-storage-and-scalability.md · 32-user-study-protocol.md
اتصال و RAG	documentation/34-external-connectors.md · 37-web-search-enrichment.md · 38-rag-architectures.md · 39-postgres-database-connector.md
اجرا و ظاهر	documentation/33-docker-and-local-llm.md · 40-production-deployment.md · 41-internationalization-and-fonts.md

روش بررسی: خواندن دستور پروژه و فهرست ویکی، مرور ۴۱ راهنما و خواندن بخش‌های مرتبط، تطبیق مسیرهای رابط و API با کد، بررسی قرارداد تنظیمات و Compose، و شمارش موجودی فایل‌ها. فایل محرمانهٔ env. و دادهٔ حساب و نشست خوانده نشده و هیچ سرویس زنده یا مدل واقعی برای ارزیابی برنامه فراخوانی نشده است.

> **کاربرد این نسخه**
>
> برای شناخت و تحویل فنی پروژه قابل استفاده است. با تغییر مخزن، مسیرها، تعداد فایل‌ها و مرز قابلیت‌ها باید دوباره تطبیق داده شوند. ادعاهای کیفیت، هزینه و سود کاربری به شواهد آزمایشی مستقل نیاز دارند.

Sources: AGENTS.md | README.md | documentation/README.md | .github/workflows/pr-checks.yml